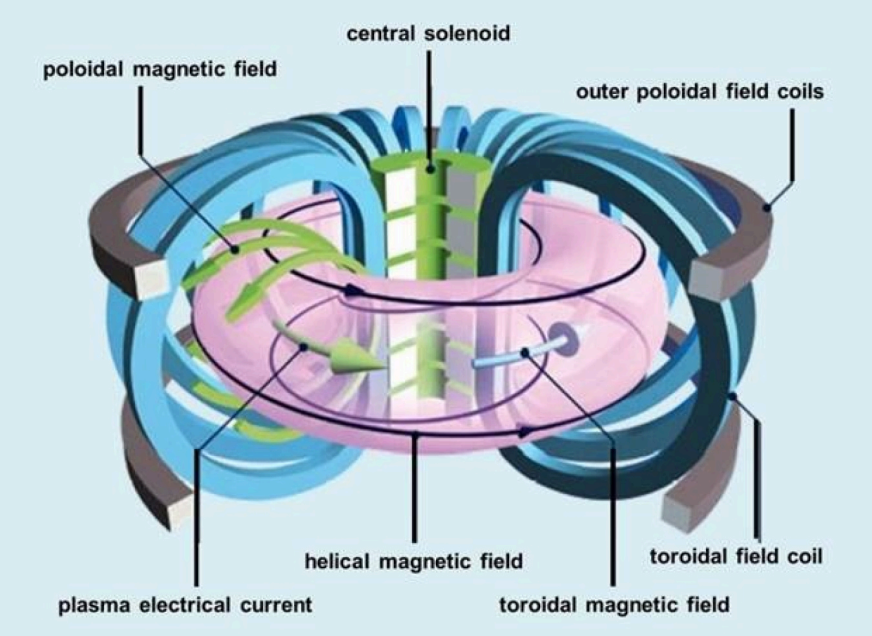
CONFERENCE ON COMPLEX SYSTEMS 2026 · PROJECT REPORT

Generative AI for Scalable and Structure-Preserving Plasma Simulations

A Complex Systems Approach to Nuclear Fusion Energy
Everything we did, explained from scratch for a non-specialist reader

Sarthak Sharma (PhD student) and **Matthew Knepley**
State University of New York at Buffalo (University at Buffalo), Buffalo, NY, USA
ss694@buffalo.edu



Schematic of a tokamak and its magnetic fields (image supplied by the author; explained in Section 2.2).

Companion to the 12-minute virtual talk at CCS 2026 (13 October 2026). Report compiled 12 October 2026 (ET).
Computing: Argonne Leadership Computing Facility (ALCF) Director's Discretionary allocation **DD_plasma_physics** on the Aurora and Polaris supercomputers, Argonne National Laboratory.
All numbers in this report come from the project's own result files (listed in Appendix A). Nothing is extrapolated beyond what was measured.

Contents

1 Executive summary 3
What we found, in plain words 3
How to read this report 5
2 Background primer: everything you need, from scratch 6
2.1 Plasma and nuclear fusion 6
2.2 Tokamaks: a magnetic bottle 7
2.3 Charged particles in magnetic fields: the Lorentz force 8
2.4 Computers step forward in time: numerical integrators and error accumulation 8
2.5 Conservation laws and invariants 9
2.6 Hamiltonian structure and symplectic integrators, intuitively 10
2.7 Fields on grids: staggered grids and constrained transport 11
2.8 Kinetic plasma simulation: particle-in-cell and Landau damping 11
2.9 Supercomputers, MPI ranks and scaling 12
2.10 PETSc: the toolkit 14
2.11 AI agents, LLMs and MCP servers, and why verification matters 14
3 The problem and where we started 16
3.1 The two coupled challenges 16
3.2 What existed before this phase of work 16
4 Methods and how the work was organised 17
5 Experiment 1: pushing a single charged particle for a million steps 18
5.1 Question 18
5.2 Setup 18
5.3 Results 18
5.4 What it means 20
6 Experiment 2: keeping $\nabla \cdot \mathbf{B} = 0$ on a grid 21
6.1 Question 21
6.2 Setup 21
6.3 Results 21
6.4 What it means 22
7 Experiment 3: a kinetic plasma on two supercomputers 23
7.1 Question 23
7.2 Setup 23
7.3 Results: physics and conservation 23
7.4 Results: did GPUs help? 24
8 The AI-agent part: generating code that proves it is correct 25
8.1 The architecture 25
8.2 Earlier result: agent-generated, verified Grad–Shafranov solvers 26
8.3 New in this project: the VERIFY{} conservation contract 27
9 Scaling on supercomputers 29
9.1 First pilot: the code did not scale, and particles went missing 29
9.2 Diagnosis: one rank's search grid covered the whole domain 29
9.3 The fix 30
9.4 Weak scaling after the fix 30
9.5 What the numbers say, on the same basis 32
9.6 Conservation at every scale 32
10 Phase 4: stronger tests of scale and of time 34
10.1 Strong scaling: one fixed problem on more nodes 34
10.2 Weak scaling to 64 Aurora nodes 36
10.3 Long runs of the kinetic plasma: numerical heating 37
10.4 Particle pushers for 100 million steps 38
10.5 Summary of Phase 4 40
11 Compute used and acknowledgement 40
12 Limitations: an honest list 41
13 Next steps 42
14 Glossary 43
15 References 45
Appendix A. Where every number comes from 45

1 Executive summary

The big picture. Nuclear fusion, the process that powers the Sun, could one day give us clean electricity. The leading machine design, the *tokamak*, holds a gas of electrically charged particles (a *plasma*) in a doughnut-shaped magnetic "bottle". Designing such machines relies heavily on computer simulation. Simulating a plasma is hard: billions of particles interact through electric and magnetic fields, on time scales from fractions of a nanosecond to seconds. A long simulation may need a million or more time steps, and small numerical errors made at each step can pile up into results that are physically impossible, for example a plasma that slowly gains energy from nowhere.

What this project asks. The project has two connected goals, stated in the CCS 2026 abstract:

1. **Structure-preserving simulation.** Use numerical methods that respect the physics "by construction". They keep the quantities nature conserves (energy, momentum, the volume of "phase space", and the rule that magnetic fields have no sources, $\nabla \cdot \mathbf{B} = 0$) exactly or within tight bounds, instead of letting errors accumulate. These methods are built inside **PETSc**, a widely used open-source library for scientific computing.
2. **AI that writes simulation code and checks it.** Build a team of cooperating AI "agents" (programs driven by large language models) that turn a plain-language physics request into working, verified C code for PETSc. A key requirement is that every generated program must *prove* it conserves what it should, instead of the AI simply being trusted.

A third strand is to show that the resulting simulations run efficiently on **Aurora** and **Polaris**, two supercomputers at Argonne National Laboratory.

What we found, in plain words

- **The standard method drifts. The structure-preserving methods do not.** We followed one charged particle for one million time steps in three test fields. The classic fourth-order Runge–Kutta method (RK4) let the particle's energy drift by **19 %** in a tokamak-like (Solov'ev) magnetic field and by **52 %** in a uniform magnetic field. The Boris method and an energy-conserving "discrete gradient" method (PETSc's `TSDISCGRAD`) kept both energy and angular momentum bounded for all 10^6 steps. `TSDISCGRAD` held the energy to between 5×10^{-13} and 5×10^{-10} of its starting value.
- **"Symplectic" is not a magic word.** PETSc's built-in symplectic integrator (`TSBASICSYMPLECTIC`) assumes the energy splits neatly into a kinetic part and a potential part. A magnetic force breaks that assumption. In magnetic fields the method blew up in under 2,000 steps. The method has to match the structure of the physics.
- **Magnetic fields stay source-free to round-off.** A "constrained transport" solver on a staggered grid (PETSc `DMSTAG`) kept the discrete $\nabla \cdot \mathbf{B}$ at 3×10^{-24} (essentially zero) for the whole run. A conventional cell-centred method reached about 5×10^{-2} within the first steps. Both are equally accurate (second order) for smooth fields.
- **A kinetic plasma simulation gives the same answer on both supercomputers.** PETSc's particle-in-cell example of *Landau damping* gave identical results on Aurora and Polaris and at every rank count tested. The measured damping rate was −0.161, against −0.1533 from theory.

- **We found and fixed a real scaling bottleneck.** When run on many processors, one processor (the one owning the "wrap-around" cell of the periodic domain) did up to **41×** more point-location work than the others, and everyone waited for it. A 27-line local patch fixed this. At 16 Aurora nodes (1,536 processes, 78.6 million particles) a time step became **12.2× faster**. Measured against one full node, efficiency went from **0.47 to 0.75** on Aurora (16 nodes) and from 0.19 to 0.80 on Polaris (8 nodes). Particle count and charge were conserved exactly at every scale.
- **New in Phase 4: the code keeps scaling, to 64 Aurora nodes.** With the same work per process, the code ran on 64 Aurora nodes (6,144 processes, 315 million particles) at **58 %** of single-node efficiency, with no particle lost. Efficiency falls gradually: 1.00, 0.90, 0.87, 0.82, 0.75, 0.63 and 0.58 at 1, 2, 4, 8, 16, 32 and 64 nodes. Splitting one fixed 9.83-million-particle problem over more nodes ("strong scaling") made it 4.63× faster on 16 Aurora nodes and 5.38× faster on 8 Polaris nodes. Aurora runs out of work per process at that size.
- **New in Phase 4: long runs separate the methods even more clearly.** Over 10^8 steps, 100× longer than before, Boris stayed bounded (energy error 1.2×10^{-12} in the Solov'ev field) while RK4 drifted by 28 % (Solov'ev) and 67 % (uniform field). The kinetic plasma simulation, however, slowly *heats up* in long runs: its energy error is tiny while the plasma wave damps, but then grows steadily (0.4 % by $t = 300$, and about 80 % by $t = 3000$ with few particles). This is a known weakness of standard particle-in-cell methods, and it is now measured evidence for the planned energy-conserving version.
- **AI agents can produce verified solvers.** In earlier work (July–August 2026) the multi-agent system generated a tokamak equilibrium (Grad–Shafranov) solver with zero hand-written lines. It was verified to second-order accuracy, and its safety-factor profile matched an independent code (FreeGS) to within 0.2 %. In this project we added a machine-readable `VERIFY{…}` "conservation contract". In an offline test it correctly passed Boris and `TSDISCGRAD` and failed RK4 and the unstable integrator.

19 % / 52 %
RK4 energy drift after 10^6 steps (Solov'ev / uniform B). Boris and discrete gradient stay bounded.

3×10^{-24}
max discrete ∇·B with constrained transport, vs ≈5×10^{-2} for a cell-centred baseline

12.2×
faster per step at 16 Aurora nodes after fixing the wrap-rank load imbalance

0.75 / 0.58
Aurora weak-scaling efficiency vs 1 node at 16 / 64 nodes (16 nodes before the fix: 0.47)

315 M
particles at 64 Aurora nodes (6,144 ranks); no particle lost in any run

≈ 19.1
node-hours used by the whole project on ALCF (Phases 2–4; Phase 4 ≈ 5.5)

WHAT WE DID NOT (YET) DO

Honesty matters in a project about verification, so here is what was *not* achieved. The AI verification contract was tested offline only: no end-to-end loop with a live language model was run in this phase. The divergence-free test is 2-D and "kinematic" (the flow is prescribed). The kinetic simulation is one-dimensional in space and velocity (1D-1V), and the scaling runs are only 100 steps long. Long runs show that the kinetic simulation's total energy is **not** bounded: it slowly heats up (Section 10.3). The 10^8-step run of the discrete-gradient pusher stalled and gave no long-time result. The ALCF runs used CPUs. The GPU path we tried was slower, because the particle work stays on the CPU. The weak-scaling study reached 64 Aurora nodes, far short of the "thousands of GPU nodes" targeted in the abstract. The load-imbalance fix is a local patch to one PETSc example and has not been sent upstream. Section 12 has the full list.

How to read this report

Section 2 is a self-contained primer: it explains, from scratch, every idea needed later (plasma, tokamaks, magnetic forces on particles, how computers step equations forward in time, conservation laws, symplectic methods, grids, particle-in-cell, supercomputers, PETSc, and AI agents). Readers who know some of this can skip ahead. Section 3 states the problem and the gaps we started from. Section 4 explains how the work was organised. Sections 5–7 present the three numerical experiments, Section 8 the AI-agent work and Section 9 the supercomputer scaling study, and Section 10 the Phase 4 follow-up (strong scaling, 64-node weak scaling and very long runs), each in the same format: *the question, the setup, the result, and what it means*. Section 11 lists the compute used, and Sections 12–13 cover limitations and next steps. A glossary (Section 14) defines every technical term, Section 15 lists references, and Appendix A lists where every number came from. Coloured boxes mark key ideas (green), plain-language explanations (blue), "what it means" (violet) and caveats (orange).

2 Background primer: everything you need, from scratch

This section assumes only high-school physics and some calculus. Each subsection builds on the previous ones.

2.1 Plasma and nuclear fusion

Atoms, ions and plasma. An ordinary atom has a positively charged nucleus surrounded by negatively charged electrons, so overall it is neutral. Heat a gas enough and collisions knock electrons off the atoms. You are left with a mixture of free electrons (negative) and *ions*, the nuclei that lost their electrons (positive). This electrically charged gas is a **plasma**, often called the fourth state of matter. Because its particles are charged, a plasma responds strongly to electric and magnetic fields. The moving particles also *create* fields, which in turn push the particles. This two-way feedback makes plasmas behave in rich, collective ways: waves, instabilities and turbulence. That is why the abstract calls plasma physics "one of the richest and most challenging instances of complex systems science: a high-dimensional, nonlinear, multiscale system."

Fusion. Two light nuclei that collide hard enough can merge (*fuse*) into a heavier one, releasing energy. Both nuclei are positive and repel each other, so they must hit at enormous speeds. In practice that means the fuel must be an extremely hot plasma. The reaction most studied for power plants fuses two forms of hydrogen, deuterium (one proton and one neutron) and tritium (one proton and two neutrons). It produces a helium nucleus, a free neutron and energy (Figure 1). In a power plant the released energy would be captured as heat, which boils water and drives a turbine, as in other thermal power stations (Figure 2).

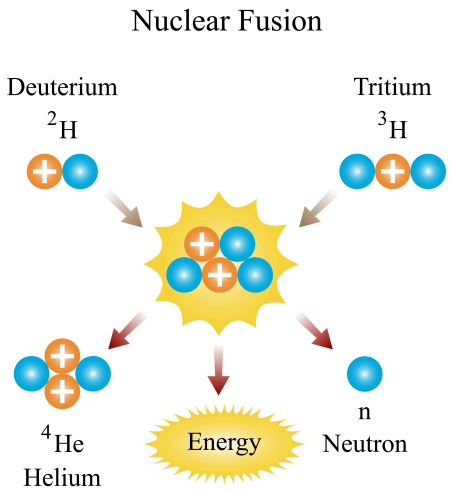


Figure 1. Deuterium–tritium fusion. Orange balls are protons and blue balls are neutrons. Look at the bookkeeping: 2 + 3 = 5 particles go in, and a 4-particle helium nucleus plus 1 free neutron come out, with energy released. (Image supplied by the author.)

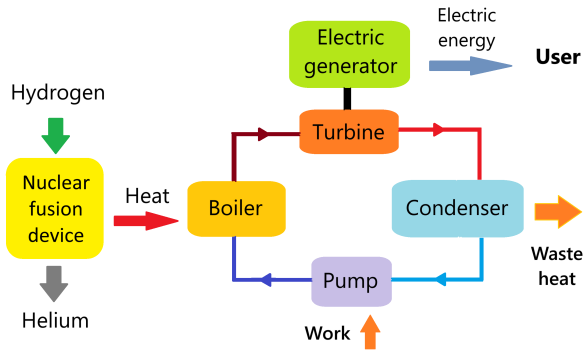


Figure 2. How a fusion power plant would make electricity. Follow the arrows. The fusion device turns hydrogen fuel into helium and heat. The heat boils water in a conventional steam loop (boiler → turbine → condenser → pump), and the turbine drives an electric generator. The plasma physics this project simulates all happens inside the yellow box on the left. (Image supplied by the author.)

2.2 Tokamaks: a magnetic bottle

No material wall can touch a fusion plasma without cooling it (and being damaged). Instead, the plasma is held by magnetic fields. A **tokamak** is a doughnut-shaped (toroidal) vacuum chamber surrounded by magnetic coils (title page figure). Two field directions matter:

- The **toroidal** field runs the long way around the doughnut. It comes mostly from the large "D-shaped" coils.
- The **poloidal** field runs the short way, around the doughnut's cross-section. It comes mostly from an electric current driven through the plasma itself (induced by the central solenoid) and from the outer poloidal field coils.

Together they make **helical** field lines that wind around the torus like the stripes on a candy cane. Charged particles tend to follow field lines (Section 2.3), so the helical winding keeps them circulating inside the chamber instead of drifting into the wall.

Coordinates. Tokamak physicists use cylindrical coordinates (R, φ, Z). Here R is the distance from the central vertical axis, Z is the height, and φ is the angle around the axis (the toroidal direction). A tokamak is close to *axisymmetric*: it looks the same at every angle φ. That symmetry will give us a conserved quantity (Section 2.5).

Equilibrium and the Grad–Shafranov equation. In a steady state the outward push of the hot plasma's pressure must be balanced by magnetic forces. For an axisymmetric tokamak this balance reduces to a single equation for one function, the **poloidal magnetic flux** $\psi(R, Z)$. You can think of ψ as a "height map" whose contour lines are the nested **flux surfaces** on which field lines lie. The **Grad–Shafranov equation** is

$$\Delta^* \psi \;\equiv\; \frac{\partial^2 \psi}{\partial R^2} - \frac{1}{R}\frac{\partial \psi}{\partial R} + \frac{\partial^2 \psi}{\partial Z^2} \;=\; -\mu_0 R^2 \frac{dp}{d\psi} - F\frac{dF}{d\psi}.$$

> $\psi(R, Z)$: poloidal magnetic flux (the unknown). Δ^*: the Grad–Shafranov differential operator defined by the middle expression.
>
> $p(\psi)$: plasma pressure, constant on each flux surface. $F(\psi) = R\,B_\varphi$: related to the toroidal field B_φ. μ_0: the magnetic constant (permeability of free space).

Special choices of p and F give exact formulas for ψ. These are the **Solov'ev** equilibria, used here both as a realistic tokamak-like magnetic field for particle tests (Section 5) and as exact answers for checking AI-generated solvers (Section 8). The Cerfon–Freidberg family of Solov'ev solutions can be shaped to resemble real machines (ITER-like, NSTX-like, and with magnetic "X-points").

Safety factor q. Follow one field line around the torus. The **safety factor** q is the number of times it goes around the long (toroidal) way for each time it goes around the short (poloidal) way. It generally differs from one flux surface to the next, so we speak of a *profile* $q(\psi)$. Common labels are q_0 at the centre (the magnetic axis) and q_{95} near the edge (on the surface enclosing 95 % of the flux). The profile matters because the stability of a tokamak plasma depends strongly on it, so it is a sensitive check of an equilibrium solver.

2.3 Charged particles in magnetic fields: the Lorentz force

A particle with charge q and mass m, moving with velocity $\mathbf{v}$ through an electric field $\mathbf{E}$ and a magnetic field $\mathbf{B}$, feels the **Lorentz force**

$$m\,\frac{d\mathbf{v}}{dt} \;=\; q\,(\mathbf{E} + \mathbf{v} \times \mathbf{B})\,, \qquad \frac{d\mathbf{x}}{dt} = \mathbf{v}.$$

> $\mathbf{x}$: position. $\mathbf{v}$: velocity. t: time. $\times$: the vector cross product, which gives a vector perpendicular to both $\mathbf{v}$ and $\mathbf{B}$.

The mechanism. Because $\mathbf{v} \times \mathbf{B}$ is always perpendicular to $\mathbf{v}$, the magnetic force never speeds the particle up or slows it down. It only turns it. Seen along the field, the particle runs in circles (*gyrates*) at the **gyrofrequency** (or cyclotron frequency) $\omega_c = |q|B/m$, while sliding freely along the field line. The result is a tight helix wrapped around the field line, which is why tokamak field lines guide the particles. Two consequences matter for the numerical methods later:

- With $\mathbf{E} = 0$, the speed $|\mathbf{v}|$, and hence the kinetic energy, must stay exactly constant. A good numerical method should reproduce this.
- The gyration is fast. A simulation must take steps Δt smaller than one gyration period. In our tests $\omega_c \Delta t \approx 0.2$, i.e. about 30 steps per gyration. Following a particle for many orbits therefore means very many steps, here 10^6.

2.4 Computers step forward in time: numerical integrators and error accumulation

Equations like the Lorentz force law are **ordinary differential equations** (ODEs): they say how the state $u = (\mathbf{x}, \mathbf{v})$ changes at each instant, $du/dt = f(u)$. Except in simple cases we cannot solve them with a formula, so a computer approximates the solution by taking many small time steps of size Δt. A recipe for one step, $u^n \to u^{n+1}$, is a **time integrator** (or time-stepping scheme). Here u^n means the approximate state at time $t_n = n\Delta t$.

The simplest example (forward Euler): $u^{n+1} = u^n + \Delta t\, f(u^n)$. Assume the current rate of change holds for a whole step. It is easy to see why this makes an error. If you are moving in a circle and always step along the current tangent, you step slightly *outward* every time. After many steps you spiral outward, and the "energy" of the circular motion grows without bound, even though the true motion keeps it constant.

Runge–Kutta 4 (RK4). The classic fourth-order Runge–Kutta method samples the rate of change four times per step and combines the samples cleverly:

$$k_1 = f(u^n), \quad k_2 = f\!\left(u^n + \tfrac{\Delta t}{2} k_1\right), \quad k_3 = f\!\left(u^n + \tfrac{\Delta t}{2} k_2\right), \quad k_4 = f(u^n + \Delta t\, k_3)\,,$$
$$u^{n+1} = u^n + \tfrac{\Delta t}{6}\,(k_1 + 2k_2 + 2k_3 + k_4)\,.$$

Its error per step is tiny: it shrinks like Δt^5, and the error at a fixed final time shrinks like Δt^4 ("fourth order"). RK4 is the workhorse of scientific computing. In PETSc it is `TSRK` with type 4.

Why small errors still accumulate. "Small error per step" is not the same as "correct long-time behaviour". RK4's tiny per-step errors are not random. They tend to push the energy consistently in the same direction (for gyration, slightly inward or outward each turn), so over a million steps they add up, a

secular drift. A method can be very accurate over one orbit and still produce a qualitatively wrong answer over a million orbits, for example a particle slowly spiralling out of its confining field. This is the central problem the project addresses.

> **KEY IDEA**
>
> Accuracy (small error per step) and fidelity (correct long-time behaviour) are different properties. For long plasma simulations, fidelity comes from preserving the *structure* of the equations, the quantities they conserve, rather than from making each step more accurate.

Order of accuracy, p. We will often quote an observed order p. It means the error scales like h^p, where h is the step size or grid spacing. If $p = 2$, halving h cuts the error by $2^2 = 4$. Measuring p on a problem with a known answer, and checking that it matches the theoretical order, is a standard way to *verify* that code implements its method correctly.

2.5 Conservation laws and invariants

An **invariant** is a quantity that the exact equations keep constant forever. The ones in this project are:

- **Energy.** For a particle in a static field, the total energy is kinetic plus potential, $H = \frac{1}{2}|\mathbf{v}|^2 + \frac{q}{m}\phi(\mathbf{x})$ (per unit mass, in the scaled units of our code). Here ϕ is the electric potential, with $\mathbf{E} = -\nabla\phi$. The magnetic force does no work, so it does not appear in H. For a whole plasma, the total energy is the particles' kinetic energy plus the energy stored in the fields.
- **Momentum, via symmetry.** A deep result (Noether's theorem) says every symmetry gives a conservation law. A tokamak field is the same at every toroidal angle φ, so a particle's **canonical toroidal angular momentum** is conserved: $p_\varphi = R\,v_\varphi + \frac{q}{m}\psi(R, Z)$. Here v_φ is the toroidal velocity and ψ the flux function of Section 2.2. This invariant is what keeps particles confined: a particle cannot drift far across flux surfaces without changing p_φ.
- **Phase-space volume.** "Phase space" is the space of all positions *and* velocities, six numbers per particle. Picture a small cloud of nearby starting states. As each state evolves, the cloud stretches and twists, but for these equations its six-dimensional volume never changes (Liouville's theorem). A method that shrinks or grows phase-space volume behaves like artificial friction or artificial heating. We measure this with the **Jacobian determinant** of the numerical map: how much a tiny box of starting states is scaled in volume by 100 steps. The exact value is $\det J = 1$, so we report $\det J - 1$.
- **No magnetic monopoles: $\nabla \cdot \mathbf{B} = 0$.** Magnetic field lines never begin or end. They form closed loops or wander forever. Mathematically, the *divergence* of $\mathbf{B}$ (the net amount of field "flowing out" of a tiny box) is zero everywhere. If numerical error makes it non-zero, the simulation contains fake magnetic charges that exert unphysical forces.
- **Particle number and charge.** In a particle simulation with no sources or sinks, the number of particles and their total charge must stay fixed. That sounds trivial, but on a parallel computer particles move between processors, and a bug can silently lose them (Section 9.1).

2.6 Hamiltonian structure and symplectic integrators, intuitively

Many physical systems, including charged particles in fields, are **Hamiltonian**. Their whole motion is generated by one energy function H through a fixed geometric rule. A general way to write this is

$$\frac{du}{dt} = S(u)\,\nabla H(u),$$

> u: the state (positions and velocities). ∇H: the gradient of the energy, the direction in which energy increases fastest. S: a *skew-symmetric* matrix ($S^\top = -S$). Skew-symmetry is the key: it turns the motion sideways to the energy gradient, so $dH/dt = \nabla H \cdot S\nabla H = 0$ and energy is automatically conserved.

The rule also guarantees conservation of phase-space volume (and of a finer geometric property called *symplecticity*). Structure-preserving integrators copy one of these properties exactly at the discrete level, step by step:

- **Symplectic integrators** (e.g. Störmer–Verlet, also called leapfrog). They do not conserve energy exactly, but they exactly conserve a slightly perturbed "shadow" energy. The true energy then oscillates within a bounded band forever instead of drifting. Störmer–Verlet works when the energy splits into a part depending only on velocity and a part depending only on position, $H = T(\mathbf{v}) + V(\mathbf{x})$, called a **separable** Hamiltonian:

  $$\mathbf{v}^{n+\frac{1}{2}} = \mathbf{v}^n + \tfrac{\Delta t}{2}\,\mathbf{a}(\mathbf{x}^n), \quad \mathbf{x}^{n+1} = \mathbf{x}^n + \Delta t\,\mathbf{v}^{n+\frac{1}{2}}, \quad \mathbf{v}^{n+1} = \mathbf{v}^{n+\frac{1}{2}} + \tfrac{\Delta t}{2}\,\mathbf{a}(\mathbf{x}^{n+1}),$$

  where $\mathbf{a}$ is the acceleration. The scheme relies on $\mathbf{a}$ depending only on position. With a magnetic field, $\mathbf{a} = \frac{q}{m}(\mathbf{E} + \mathbf{v} \times \mathbf{B})$ depends on velocity too, the energy is no longer separable in $(\mathbf{x}, \mathbf{v})$, and the guarantee is lost. PETSc's `TSBASICSYMPLECTIC` is of this type.
- **The Boris pusher**, the standard particle-pushing method in plasma codes. It handles the magnetic part as an exact *rotation* of the velocity, sandwiched between two half "kicks" from the electric field:

  $$\mathbf{v}^- = \mathbf{v}^n + \tfrac{q\Delta t}{2m}\mathbf{E}, \qquad \mathbf{v}^+ = \mathrm{rotate}\big(\mathbf{v}^-;\ \mathbf{B},\ \Delta t\big), \qquad \mathbf{v}^{n+1} = \mathbf{v}^+ + \tfrac{q\Delta t}{2m}\mathbf{E}, \qquad \mathbf{x}^{n+1} = \mathbf{x}^n + \Delta t\,\mathbf{v}^{n+1}.$$

  A rotation never changes the length of a vector, so with $\mathbf{E} = 0$ the speed is preserved exactly (up to round-off). The Boris method is also *volume-preserving* in phase space, though not strictly symplectic.
- **Discrete-gradient methods** (PETSc `TSDISCGRAD`; we use the Gonzalez discrete gradient). They replace ∇H in $du/dt = S\nabla H$ by a carefully averaged "discrete gradient" $\overline{\nabla} H(u^n, u^{n+1})$. It is chosen so that $H(u^{n+1}) - H(u^n) = \overline{\nabla} H \cdot (u^{n+1} - u^n)$ holds exactly. Combined with skew-symmetric S, energy is conserved to the accuracy of the equation solver at every step. The price: the method is *implicit*, because u^{n+1} appears on both sides. Every step requires solving a nonlinear equation (PETSc's SNES does this), which is much more expensive, and the method does not preserve phase-space volume exactly.

IN PLAIN WORDS

A structure-preserving method does not try to be more accurate at each step. It tries to make the *kind* of mistake that the physics forbids impossible. Boris cannot change a particle's speed in a pure magnetic field because rotations cannot change length. A discrete gradient cannot create energy because of how its average is defined. Constrained transport (below) cannot create magnetic monopoles because of how the grid is laid out.

2.7 Fields on grids: staggered grids and constrained transport

To compute a field like $\mathbf{B}$ we store its values on a **grid** (mesh) of small cells with spacing Δx, and replace derivatives by differences between neighbouring values (*finite differences*). The simplest choice stores every component at cell centres ("cell-centred" or "collocated"). Then the discrete divergence is not automatically zero, and errors in it can grow.

Staggered grids. An alternative stores different quantities at different places in each cell: the magnetic field component normal to each face *on that face*, and other quantities at edges or corners. In 2-D we write $\mathbf{B}$ through a **vector potential** A_z stored at cell corners (vertices):

$$B_x = \frac{\partial A_z}{\partial y}, \qquad B_y = -\frac{\partial A_z}{\partial x} \quad \Longrightarrow \quad \nabla \cdot \mathbf{B} = \frac{\partial^2 A_z}{\partial x\, \partial y} - \frac{\partial^2 A_z}{\partial y\, \partial x} = 0.$$

With A_z on corners, B_x and B_y are differences of corner values on the faces. The discrete divergence of a cell then adds and subtracts the same four corner values, which cancel *exactly*. The cancellation is algebraic and does not depend on how accurate the values are. Updating the field only through this structure is called **constrained transport (CT)**. The field evolves by the **induction equation** $\partial \mathbf{B}/\partial t = \nabla \times (\mathbf{v} \times \mathbf{B})$, where $\mathbf{v}$ here is the plasma flow velocity. In 2-D with CT this becomes the advection (transport) of A_z by the flow. A divergence-free result is then guaranteed to round-off for all time. PETSc's `DMSTAG` manages exactly this kind of staggered grid.

2.8 Kinetic plasma simulation: particle-in-cell and Landau damping

A tokamak contains far too many particles to follow individually. A **kinetic** description instead tracks the *distribution function* $f(\mathbf{x}, \mathbf{v}, t)$, the density of particles at each point of phase space. For electrons with a self-consistent electric field, the simplest model is the **Vlasov–Poisson** system:

$$\frac{\partial f}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f + \frac{q}{m} \mathbf{E} \cdot \nabla_{\mathbf{v}} f = 0, \qquad \mathbf{E} = -\nabla \phi, \qquad -\nabla^2 \phi = \frac{\rho}{\varepsilon_0}.$$

> The first equation says f is carried along particle trajectories. The last two say the charge density ρ (from f, plus a neutralising ion background) determines the potential ϕ and field $\mathbf{E}$ (Poisson's equation; ε_0 is the electric constant). In our tests space and velocity are each one-dimensional ("1D-1V").

Particle-in-cell (PIC) represents f by many computational "macro-particles", each standing for many real ones and carrying a weight. Each time step runs the cycle in Figure 3. Particles deposit their charge onto a grid, the field is solved on the grid, it is interpolated back to the particles, and the particles are pushed by the force. Finally the code works out which cell (and which processor) each particle now lives in: **point location** and **migration**. That last step turned out to be the scaling bottleneck in Section 9.

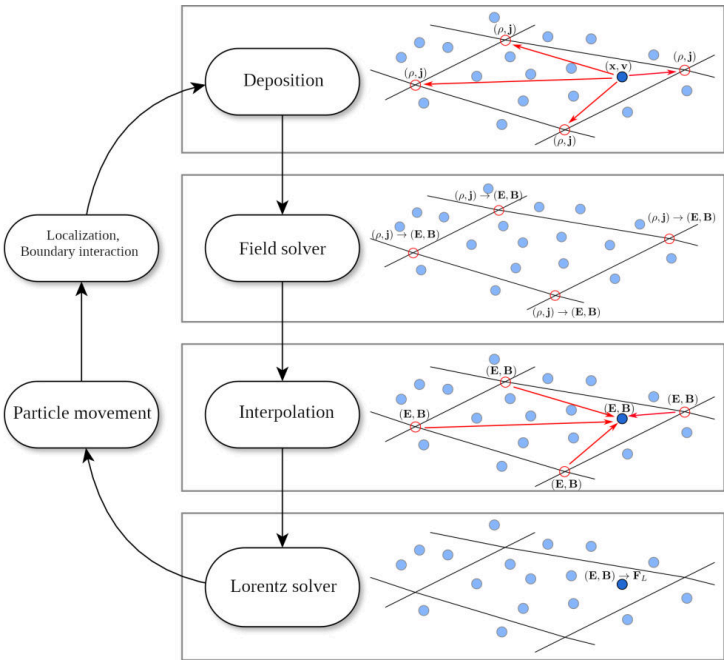


Figure 3. The particle-in-cell cycle. Read clockwise from the top. *Deposition*: each particle (dark dot at $(\mathbf{x}, \mathbf{v})$) spreads its charge ρ and current $\mathbf{j}$ to nearby grid nodes (red circles). *Field solver*: compute $\mathbf{E}, \mathbf{B}$ on the grid. *Interpolation*: bring the fields back to each particle. *Lorentz solver*: compute the force $\mathbf{F}_L$. *Particle movement*: push the particle. *Localization*: find which cell it is now in. In the electrostatic tests here only ρ and $\mathbf{E}$ are used. (Image supplied by the author.)

Landau damping is a classic kinetic test. Start with a uniform plasma, nudge its density into a small ripple $f \propto (1 + \alpha \cos kx)$ with amplitude $\alpha = 0.01$ and wavenumber $k = 0.5$, and watch the electric field. The field oscillates, and its amplitude *decays* exponentially even though there are no collisions. Particles moving at about the wave's speed exchange energy with it, and on balance they absorb it. Linear theory predicts the decay rate, $\gamma \approx -0.1533$ for $k = 0.5$ (in the code's normalised units), so it is a sharp correctness check for a PIC code.

2.9 Supercomputers, MPI ranks and scaling

Nodes, cores, GPUs. A supercomputer is many computers (**nodes**) joined by a fast network. Each node has CPUs with many **cores** and usually several **GPUs** (graphics processors, which are very fast at doing the same arithmetic on lots of data at once). This project used two machines at the Argonne Leadership Computing Facility (ALCF), Figure 4:

- **Aurora**: each node has 6 Intel Data Center GPU Max devices (12 GPU "tiles") and 2 Intel Xeon (Sapphire Rapids / Xeon Max) CPUs. We ran 96 MPI processes per node on the CPUs.
- **Polaris**: each node has 4 NVIDIA A100 GPUs (40 GB) and 1 AMD EPYC (Milan) CPU, about 560 nodes in total. We ran 32 MPI processes per node on the CPU, or 4 per node (one per GPU) for the GPU comparison.



Figure 4. The Aurora supercomputer at Argonne National Laboratory. Each cabinet in the row holds many nodes. The largest runs in this report used 64 Aurora nodes (6,144 MPI processes). (Image supplied by the author.)

MPI and ranks. Programs run in parallel by splitting the work among many independent processes that exchange messages over the network using the **Message Passing Interface (MPI)**. Each process is called a **rank** (it is numbered 0, 1, 2, …). For a simulation, the domain is cut into pieces (**domain decomposition**) and each rank owns one piece: its cells and the particles inside them. When a particle crosses into a neighbour's piece it must be sent there (**migration**).

Weak scaling and parallel efficiency. There are two ways to use more ranks. *Strong scaling* keeps the problem fixed and splits it finer (Section 10.1). **Weak scaling** grows the problem with the machine: every rank always gets the same amount of work (here 256 cells and 51,200 particles per rank). In perfect weak scaling the time per step stays constant as ranks are added. The **weak-scaling efficiency** is

$$E(P) = \frac{T(P_{\text{ref}})}{T(P)},$$

> $T(P)$: wall-clock time per step on P ranks (measured on the slowest rank). P_{ref}: the reference (baseline) run. $E = 1$ is perfect and $E = 0.5$ means each step takes twice as long as the baseline.

BASELINES MATTER

The value of E depends on the baseline. With $P_{\text{ref}} = 1$ rank, E includes the slowdown from sharing one node's memory among 96 processes, which no network can fix. With $P_{\text{ref}} =$ one full node, E isolates the cost of going *across* nodes. This report always states which baseline is used, and compares before/after only on the same baseline.

Load imbalance. Parallel programs periodically synchronise: all ranks must finish a stage before any can continue (a "collective" operation). If one rank has more work than the others, everyone waits for it, and the whole computation runs at the speed of the slowest rank. We measure imbalance as the ratio of the maximum to the minimum time (or work) across ranks for an operation. A ratio of 1 is perfect balance.

2.10 PETSc: the toolkit

PETSc (the Portable, Extensible Toolkit for Scientific Computation) is a large open-source C library for solving the equations of science and engineering in parallel with MPI, with optional GPU back ends (CUDA for NVIDIA GPUs, SYCL and Kokkos for Intel and other GPUs). The project uses PETSc 3.26.0 (our own builds) and PETSc 3.25.2 (an ALCF module on Aurora). PETSc is organised into components you combine:

Component	What it does	Used here for
`DM` (data management)	Describes the mesh or grid and how it is split among ranks	All codes
`DMDA`	Simple structured (rectangular) grids	Agent-generated Grad–Shafranov solver
`DMSTAG`	Staggered structured grids (values on vertices, faces, cells)	Constrained-transport $\nabla \cdot \mathbf{B} = 0$ solver
`DMPlex`	General unstructured meshes, with finite elements (`PetscFE`)	Field mesh of the PIC code; DMPlex GS solver
`DMSwarm`	Clouds of particles attached to a mesh, including migration between ranks	Particles of the PIC code
`TS` (time stepping)	Time integrators: `TSRK` (Runge–Kutta), `TSBASICSYMPLECTIC`, `TSDISCGRAD`, …	Pusher study, CT solver, PIC
`SNES`, `KSP`, `PC`	Nonlinear solvers, linear (Krylov) solvers, preconditioners such as `GAMG` (algebraic multigrid)	Implicit steps, Poisson field solve, GS solver
Options database, `-log_view`	Run-time options on the command line; built-in performance profiling per operation and per rank	All experiments; diagnosing the bottleneck

PETSc ships tutorial examples. One of them, `src/ts/tutorials/hamiltonian/ex2.c`, is a DMSwarm Vlasov–Poisson PIC code with symplectic time stepping. We call it **ex2**, and it is the basis of all supercomputer runs.

2.11 AI agents, LLMs and MCP servers, and why verification matters

A **large language model (LLM)** is a neural network trained on large amounts of text, including computer code. Given a prompt, it produces text, for example a C program. An **AI agent** is an LLM placed in a loop with **tools**: it can decide to call a tool (search documentation, write a file, compile, run), read the result, and continue until the task is done. A **multi-agent system** splits a big job among specialised agents coordinated by an **orchestrator** agent, much as a project lead coordinates specialists.

The **Model Context Protocol (MCP)** is a standard way to expose tools to AI agents. Each tool lives in an **MCP server**, a small program that advertises functions (e.g. “compile and run this code”) that an agent can call. **Retrieval-augmented generation (RAG)** means the agent first retrieves relevant passages (here, from PETSc documentation) and includes them in its prompt, to ground its answer in real facts rather than memory.

Why verification matters. LLMs produce plausible-looking code, but plausible is not correct. A program can compile, run and print reasonable numbers while violating the physics. For example, the RK4 code in Section 5 runs perfectly well while silently creating 19 % extra energy. Asking the model whether its code is correct does not help much. Its confidence is not evidence. The approach here is to make every generated program measure and print its own invariants (energy drift, momentum drift, $\nabla \cdot \mathbf{B}$, whether

all numbers are finite) in a fixed machine-readable format, and to accept the code only if those numbers pass tolerances. As the earlier tokamak work put it: *verification, not the model's confidence, decides acceptance.*

3 The problem and where we started

3.1 The two coupled challenges

The abstract frames two challenges. **First**, accurate long-time plasma simulation "remains computationally prohibitive and numerically fragile with standard methods". Methods like Runge–Kutta "minimize local truncation error but allow conservation violations to accumulate over millions of timesteps". The proposed answer is structure-preserving methods inside PETSc that keep phase-space volume, energy, momentum and $\nabla \cdot \mathbf{B} = 0$ at the discrete level. **Second**, writing such codes by hand is slow and expert-intensive. The proposed answer is a hierarchical multi-agent AI system: an orchestrator coordinating four specialists (mathematical modeling, numerical analysis, HPC code generation and execution, visualization/analysis), with verification (conservation checks and symplecticity tests) built into the generated code. The abstract notes that this architecture aligns with the US Department of Energy's *Genesis Mission* for automated PDE-based simulation science, and it targets weak scaling on Polaris and Aurora.

3.2 What existed before this phase of work

A read-only survey (Phase 1, 11 October 2026) recorded what already existed and what was missing:

- **PETSc MCP servers** (`gitlab.com/petsc/petsc_mcp_servers`). These are working MCP servers for an orchestrator, PDE modeling, numerical analysis (NA), code generation, compile-and-run, PETSc documentation search and RAG. The orchestrator calls them in a fixed order. The NA agent chooses a grid, discretization and integrator but has **no notion of structure preservation, conservation laws or symplecticity**. The code generator checks only that the code *compiles*, not that it is correct.
- **Tokamak agent project** (`petsc_mcp_servers_tokamak`, July–August 2026, on an Argonne CELS compute node with Claude Opus via Argonne's Argo service). The agents generated and verified **Grad–Shafranov equilibrium** solvers (steady state only). Results are in Section 8.2.
- **Earlier ALCF files**: a small symplectic toy (a harmonic oscillator), scaling scripts for PETSc's ex2 with no recovered scaling data, a DMPlex skeleton with an empty physics kernel, and a non-PETSc Python reduced-MHD study. These provided useful context but no verified structure-preserving PETSc code and no scaling data.

Abstract claim	Status at the start (Phase 1 survey)
Symplectic integrators in PETSc conserving phase-space volume, energy, momentum	Only a toy and PETSc tutorial runs; no own verified code; no long-time drift study versus RK
Divergence-free field solver ($\nabla \cdot \mathbf{B} = 0$ discretely)	Nothing existed
Orchestrator + 4 specialists	3 of 4 specialists exist upstream; the visualization/analysis agent is design-only; NA agent has no structure-preserving knowledge
Verification embedded in generated code	Only manufactured-solution order-of-accuracy checks for Grad–Shafranov; no conservation or symplecticity checks
Weak scaling on Polaris and Aurora, "thousands of GPU nodes"	No scaling data at all. Polaris has only about 560 nodes, so "thousands" is only possible on Aurora

Table 1. Gap analysis from `PLAN.md` §0.4.

4 Methods and how the work was organised

The work was organised as a research plan (`PLAN.md`) with three phases, carried out with AI assistance (the Grok Bot assistant) under Sarthak Sharma's direction. Every job submission to the ALCF supercomputers required his explicit approval.

Phase	When (ET)	What
1. Survey and plan	11 Oct 2026	Read-only survey of existing code and of ALCF software, queues and storage. Research plan with methods, agent tie-in, scaling design and milestones.
2. Methods and pilots	11–12 Oct 2026	Particle-pusher study and constrained-transport solver on the project's development machine (PETSc 3.26.0, one core). ex2 Landau-damping physics runs and a small weak-scaling slice on Aurora and Polaris (debug queue, 1–2 nodes). `VERIFY{}` contract prototype.
3. Diagnose and scale	12 Oct 2026, 00:40–02:19	Diagnosis of the point-location imbalance, a local fix, and grow-the-box weak scaling to 16 Aurora nodes and 8 Polaris nodes.
4. Scale and time	12 Oct 2026	Strong scaling of a fixed problem, weak scaling to 32 and 64 Aurora nodes, long Landau-damping runs on both machines and 10^8-step pusher runs.

Table 2. Project phases. The ALCF allocation (DD_plasma_physics) runs until 7 November 2026.

Codes written or used.

- `sp/pusher/pusher.c`: a C program on PETSc `TS` that pushes one charged particle with four methods: Boris (hand-coded), RK4 (`TSRK` 4), `TSBASICSYMPLECTIC` (Störmer–Verlet), and `TSDISCGRAD` (Gonzalez discrete gradient). It records energy and p_φ and the phase-volume Jacobian.
- `sp/ct/ct2d.py` (NumPy) and `sp/ct_stag/ct_stag.c` (PETSc `DMSTAG` + `TS`): constrained-transport induction solvers that serve as independent cross-checks of each other.
- PETSc tutorial **ex2** (DMSwarm Vlasov–Poisson PIC), run unmodified in Phase 2 and with a 27-line local patch in Phase 3.
- `petsc_mcp_servers_verify`: a local branch (`grokbot/verify-contract`, not pushed) adding the `VERIFY{}` contract to the agent servers.

How each method is judged. In every experiment we measure the invariants directly: maximum relative energy error $\max_t |H(t) - H(0)|/|H(0)|$, the same for p_φ, $\det J - 1$ for phase-space volume, $\max |\nabla_h \cdot \mathbf{B}|$ scaled by $\Delta x / \max |\mathbf{B}_0|$, and particle count and total charge. We also measure order of accuracy on problems with known answers, and wall-clock time from PETSc's `-log_view`.

5 Experiment 1: pushing a single charged particle for a million steps

5.1 Question

Over very long runs, does a standard high-accuracy integrator (RK4) keep a charged particle's energy and angular momentum, and do structure-preserving alternatives do better? Which PETSc integrators can be trusted in a magnetic field?

5.2 Setup

One particle, 10^6 time steps per run, in normalised units, on the development machine (one CPU core). Three fields of increasing realism:

1. **Electrostatic (ES)**: no magnetic field. A pendulum-like potential in x and harmonic wells in y, z. This energy is separable, so it is a fair test for `TSBASICSYMPLECTIC`. $\Delta t = 0.1$.
2. **Uniform B_z** with a radial electrostatic well, $\omega_c \Delta t = 0.2$.
3. **Solov'ev tokamak field**: an analytic tokamak equilibrium (major radius $R_0 = 1$, field $B_0 = 1$, elongation κ, central safety factor q_0), $\omega_c \Delta t \approx 0.2$, with a particle on a passing orbit (one that circulates all the way around the torus).

Measured: energy $H = \frac{1}{2}|\mathbf{v}|^2 + \frac{q}{m}\phi$; canonical toroidal momentum $p_\varphi = R v_\varphi + \frac{q}{m}\psi$, which is exactly conserved in fields (b) and (c) by axisymmetry; and $\det J_{100} - 1$, the phase-volume change of the 100-step map, computed by finite differences.

5.3 Results

Field	Method	Steps	max $\lvert\Delta H/H\rvert$	max $\lvert\Delta p_\varphi/p_\varphi\rvert$	$\det J_{100} - 1$
ES	RK4	10^6	1.0e-2 (secular)	n/a	−3.5e-6
ES	TSBASICSYMPLECTIC	10^6	1.8e-3 (bounded)	n/a	−7.3e-11
ES	Boris	10^6	1.6e-3 (bounded)	n/a	−4.0e-11
ES	TSDISCGRAD	10^6	**4.9e-13**	n/a	−1.1e-4
Uniform B	RK4	10^6	5.2e-1	1.8e-1	−1.5e-4
Uniform B	TSBASICSYMPLECTIC	< 2,000	blows up	blows up	+6.3
Uniform B	Boris	10^6	4.8e-4 (bounded)	1.8e-3 (bounded)	−5.8e-11
Uniform B	TSDISCGRAD	10^6	**6.2e-12**	**1.4e-12**	+6.3e-10
Solov'ev	RK4	10^6	1.9e-1 (secular)	1.5e-4 (secular)	−5.2e-5
Solov'ev	TSBASICSYMPLECTIC	< 2,000	blows up	blows up	+4.4
Solov'ev	Boris	10^6	**1.3e-13** (E = 0)	4.6e-6 (bounded)	−3.3e-8
Solov'ev	TSDISCGRAD	10^6	4.6e-10 (≈√t, solver tol.)	2.8e-6 (bounded)	−4.3e-6

Table 3. Particle-pusher results (`RESULTS.md` §1). Notation: 1.9e-1 means $1.9 \times 10^{-1} = 19\,\%$. "Secular" means growing steadily with time, "bounded" means oscillating within a fixed band.

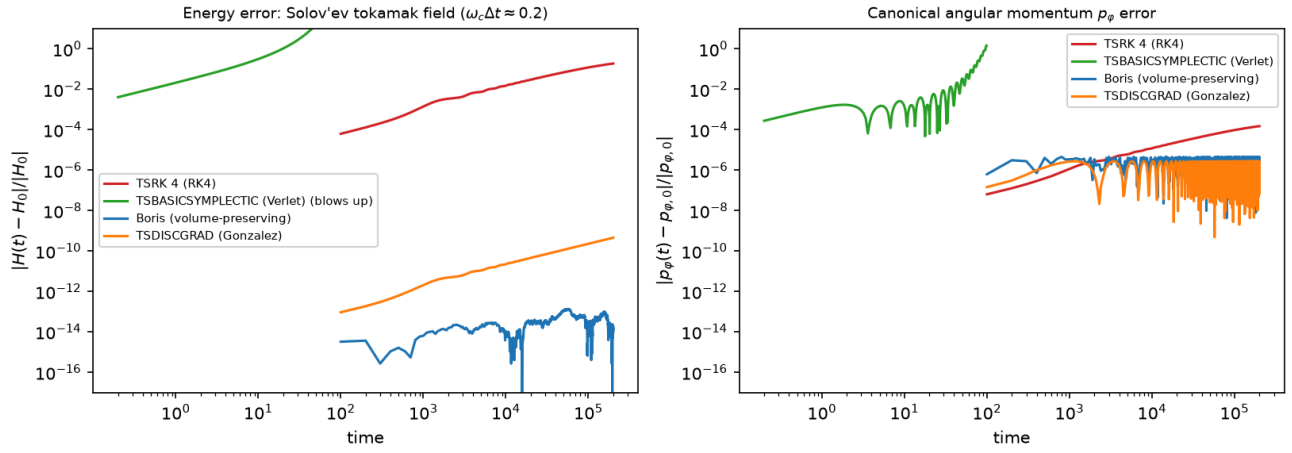


Figure 5. Energy error (left) and p_φ error (right) versus time in the Solov'ev tokamak field. Both axes are logarithmic, so equal distances mean equal factors of 10. *What to look at:* the red RK4 line keeps climbing in both panels (secular drift) and reaches about 0.19 in energy at the end. The green `TSBASICSYMPLECTIC` line shoots off the top before time 100 (blow-up). The blue Boris line stays near 10^{-14} to 10^{-13} in energy (round-off level, because $\mathbf{E} = 0$ here) and flat in p_φ. The orange `TSDISCGRAD` line grows only slowly in energy (about $\sqrt{t}$, set by the nonlinear-solver tolerance) and stays flat in p_φ.

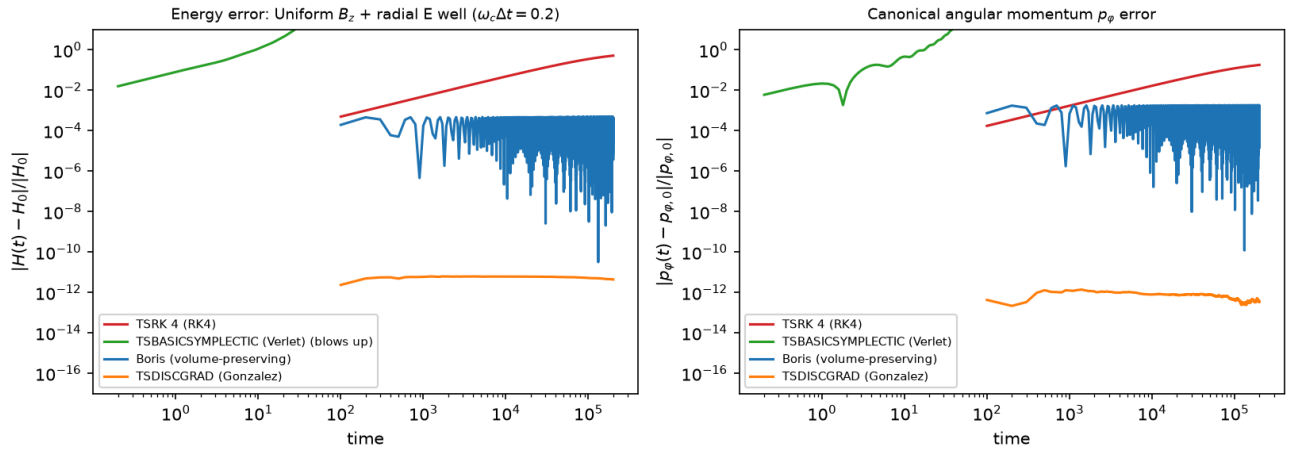


Figure 6. The same diagnostics in a uniform magnetic field with a radial electric well. *What to look at:* RK4 (red) climbs to about 0.5 in energy, the 52 % drift. Boris (blue) oscillates in a fixed band (up to about 5×10^{-4}) without growing, which is the bounded-error signature. `TSDISCGRAD` (orange) is flat near 10^{-12} in both energy and p_φ. `TSBASICSYMPLECTIC` (green) again leaves the plot early.

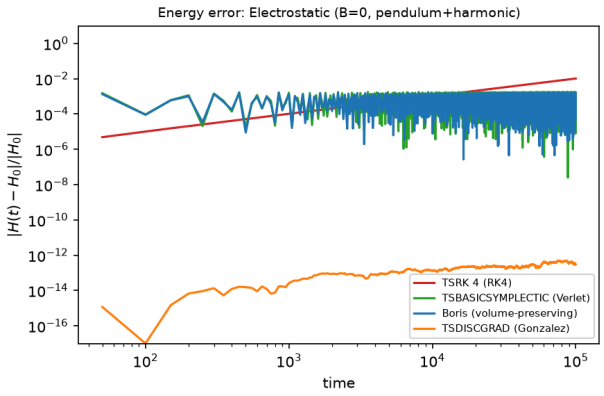


Figure 7. Energy error with no magnetic field (separable case). *What to look at:* here `TSBASICSYMPLECTIC` (green) behaves well. It overlaps Boris (blue) in a bounded band around 10^{-3}, while RK4 (red) rises as a straight line on the log–log plot (steady drift) and crosses above the band near the end. `TSDISCGRAD` (orange) sits near 10^{-13}.

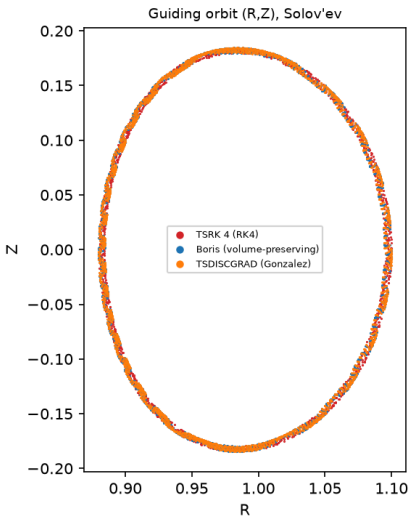


Figure 8. The particle's path projected onto the (R, Z) cross-section of the Solov'ev tokamak. *What to look at:* all three surviving methods trace the same closed oval (a passing orbit on a flux surface). A drift in invariants is a slow, long-term effect and does not show up in a single picture of the orbit shape. That is why the invariants must be measured directly.

Cost (one core, 10^6 steps): Boris 0.05 s (hand-coded), RK4 0.9 s, `TSDISCGRAD` 240–355 s, because it solves a nonlinear equation (SNES) at every step.

5.4 What it means

WHAT IT MEANS

- **RK4 drifts in both invariants.** After 10^6 steps the energy is off by **19 % in the Solov'ev field** and **52 % in the uniform field**, and p_φ drifts too. In a real simulation that would mean particles gaining energy and slowly drifting across the field: unphysical heating and loss.
- **Boris and `TSDISCGRAD` keep both invariants bounded** (or at the solver tolerance) over 10^6 gyro-steps. Boris is also by far the cheapest.
- **A trap: PETSc's `TSBASICSYMPLECTIC` assumes a separable energy $H = T(p) + V(q)$.** With $\mathbf{v} \times \mathbf{B}$ the energy is not separable in $(\mathbf{x}, \mathbf{v})$, so the method is not symplectic there. It blows up within 2,000 steps, and $\det J - 1 = O(1)$ (+6.3, +4.4) confirms it. It works for electrostatic (Vlasov–Poisson) problems, which is exactly what PETSc's ex2 uses. Lesson: the method must match the Hamiltonian structure of the problem.
- **Energy versus volume trade-off.** `TSDISCGRAD` conserves energy but not phase-space volume ($\det J - 1 = -1.1 \times 10^{-4}$ for the pendulum), while Boris preserves volume ($\sim 10^{-11}$) but conserves energy only within a bounded band when $\mathbf{E} \neq 0$. No single method keeps every invariant.

CAVEAT

One particle and one orbit class per field. `TSDISCGRAD`'s accuracy is limited by the solver tolerances (relative 10^{-12}, absolute 10^{-14}).

6 Experiment 2: keeping $\nabla \cdot \mathbf{B} = 0$ on a grid

6.1 Question

Can a grid-based magnetic-field solver keep $\nabla \cdot \mathbf{B} = 0$ to machine precision for the whole run, and does that cost any accuracy?

6.2 Setup

2-D ideal induction equation on a periodic square box, 128×128 cells. The test is the standard **field-loop advection** problem (a ring-shaped magnetic field carried by the flow), with uniform flow and with rigid rotation, using a second-order upwind (MUSCL with the minmod limiter) scheme. Two independent CT implementations were built: a NumPy prototype and a PETSc `DMSTAG` + `TS` code (time stepping with SSP-RK2; A_z on vertices, $\mathbf{B}$ on faces, $\nabla \cdot \mathbf{B}$ evaluated on cells). The baseline stores $\mathbf{B}$ at cell centres and advects it with the same scheme.

6.3 Results

- CT: $\max |\nabla_h \cdot \mathbf{B}|\, \Delta x / \max |\mathbf{B}_0| \le 2.2 \times 10^{-16}$ (NumPy) and 3.2×10^{-24} (DMSTAG) for the whole run. The cell-centred baseline reaches 4.9–5.1×10^{-2} within the first steps.
- Cross-check: the DMSTAG and NumPy CT codes give the same magnetic energy ($1.402977 \times 10^{-7} \to 1.273035 \times 10^{-7}$) to 7 digits.
- Smooth-field convergence, $N = 32 \ldots 256$ cells per side: observed order $p = 2.00,\ 2.00,\ 2.00$. For a smooth field CT and the baseline have the same error.

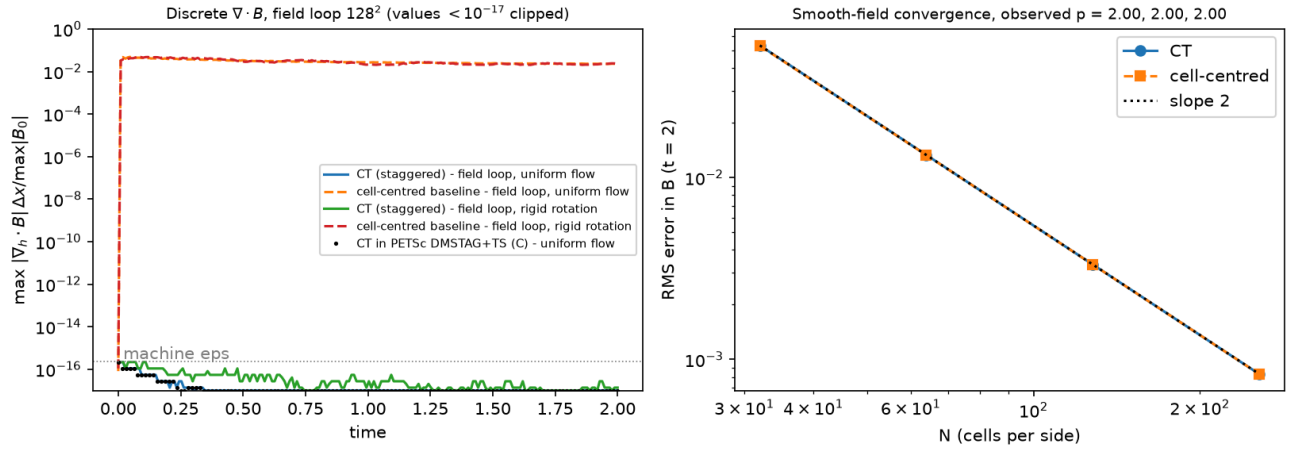


Figure 9. Left: the largest discrete divergence of $\mathbf{B}$ over time. Right: error versus grid size for a smooth field. *What to look at (left):* the dashed orange and red baseline curves jump to about 5×10^{-2} almost immediately and stay there. The CT curves (blue, green and black dots for the PETSc DMSTAG run) stay at or below the grey "machine eps" line, i.e. zero to the precision of computer arithmetic. Values below 10^{-17} are clipped to the bottom of the plot. *Right:* CT (blue) and the cell-centred baseline (orange) lie on top of each other and on the dotted slope-2 line. Both are second-order accurate, so the guarantee costs no accuracy.

6.4 What it means

WHAT IT MEANS

Because of the staggered layout, a divergence-free field cannot become non-zero. The conventional method creates spurious "magnetic charge" at the 5 % level from the very start. The structure-preserving method removes this error at no accuracy cost.

CAVEAT

2-D **kinematic** induction only: the velocity is prescribed, and there is no feedback from the magnetic force on the flow (that would be full MHD). The DMSTAG version has been run serially only.

7 Experiment 3: a kinetic plasma on two supercomputers

7.1 Question

Does a structure-preserving PIC simulation (PETSc ex2: DMSwarm particles, a finite-element Poisson solve, and `TSBASICSYMPLECTIC`, which is valid here because the problem is electrostatic) reproduce Landau damping correctly? Does it give the same answer on Aurora and Polaris and at any number of ranks? Do GPUs help?

7.2 Setup

1D-1V Landau damping: $k = 0.5$, $\alpha = 0.01$, periodic $x \in [0, 4\pi]$, $v \in [-6, 6]$, with a P1 (piecewise-linear) finite-element Poisson solve using GAMG. Each rank gets 256 x-cells $\times$ 200 v-cells $=$ **51,200 particles**. Physics runs: 600 steps to $t = 30$ with $\Delta t = 0.05$. All runs were in the ALCF debug queue on 1–2 nodes and used the CPU code path (Polaris 32 ranks per node, Aurora 96 per node), except for the explicit GPU trials.

7.3 Results: physics and conservation

Machine	Ranks	Particles	Fitted γ (theory −0.1533)	max $\lvert\Delta W/W_0\rvert$	max $\lvert\Delta W\rvert$ / peak field energy
Polaris	1	51,200	−0.1613	1.37e-5	6.8 %
Polaris	4 / 8 / 16	0.2 M / 0.4 M / 0.8 M	−0.1612 (all)	1.30e-5	6.5 %
Aurora	1	51,200	−0.1613	1.37e-5	6.8 %
Aurora	12	614,400	−0.1612	1.29e-5	6.5 %

Table 4. Landau damping with ex2 (600 steps, $t = 30$, $\Delta t = 0.05$). W is the total energy (particle kinetic plus field energy).

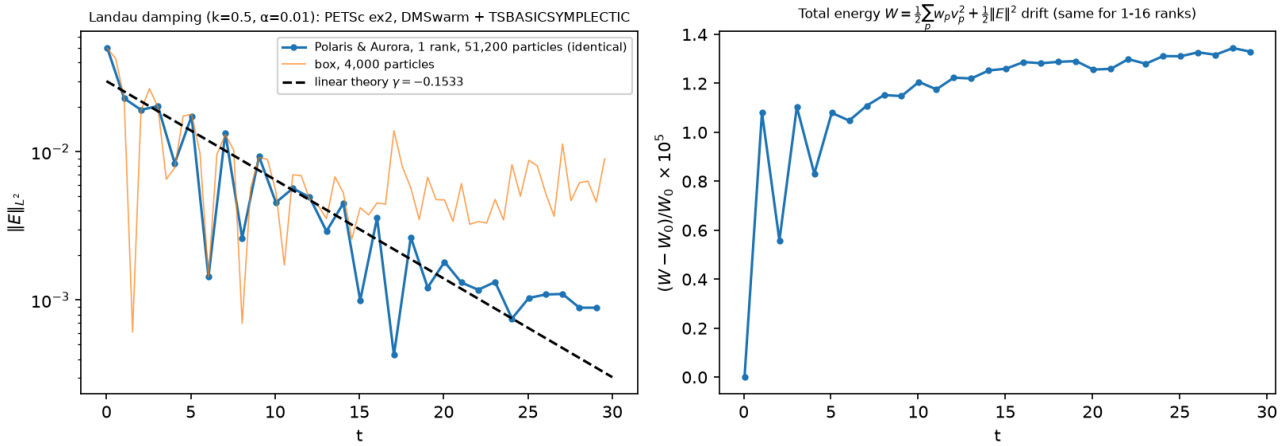


Figure 10. Landau damping in PETSc ex2. *Left:* electric-field strength $\|E\|$ (log scale) versus time. *What to look at:* the blue curve (identical on Polaris and Aurora, 1 rank, 51,200 particles) oscillates, and its peaks fall along the dashed line of slope $\gamma = -0.1533$ from linear theory. The thin orange curve, a 4,000-particle run on the development machine, follows the line at first and then flattens into noise: too few particles. *Right:* relative total-energy error $(W - W_0)/W_0$, in units of 10^{-5}. It rises quickly to about 1×10^{-5} and then creeps up to about 1.3×10^{-5}. The panel title notes that the curve is the same for 1–16 ranks.

- Aurora (1 rank, Intel oneAPI compilers) and Polaris (1 rank, GNU compilers) **match to every printed digit**. Results do not depend on the number of ranks.
- The fitted γ is about 5 % steeper than linear theory (envelope fit over $t \in [0.5, 15.5]$, 256-cell mesh). On the development machine with 4,000 particles it was −0.156.
- Total energy is **not** exactly conserved. The drift is about 10^{-5} of the total energy but 6–7 % of the peak field energy. Verlet with a finite-element-projected field is symplectic in the particle variables only while the field is frozen. With $\Delta t = 0.01$ the drift falls to 2.2×10^{-6}, about 5× smaller for a 5× smaller step, which suggests roughly first order. That is a hint, not a measured order. Much longer runs (Section 10.3) show that this error is **not bounded**: once the wave has damped, the energy grows slowly (numerical heating).

7.4 Results: did GPUs help?

Mostly no, and that is an important finding.

- **Polaris** (job 7738042, 4 ranks, 1 per A100, CUDA vectors/matrices): runs correctly, and the total-energy error is identical to the CPU run ($\Delta W/W = 2.326 \times 10^{-6}$). The Poisson solve ran 100 % on the GPU, but the run is **1.35× slower** (time-stepping 34.6 s vs 25.5 s). In ex2 only the field solve is offloaded, and it is about 1 % of the work. Particle push, location and migration stay on the CPU.
- **Aurora** (ALCF module `petsc/3.25.2-sycl`, Kokkos/SYCL back end, 1 rank on one GPU tile, job 8922008): the first attempt failed until the environment variable `ZES_ENABLE_SYSMAN=1` was set. With it, the run completes, and its final field and energy match the CPU run to 4×10^{-14} relative. But time-stepping takes **169.6 s versus 14.5 s on one CPU core (11.7× slower)**: the tiny 256-cell Poisson solve is dominated by GPU launch and data-transfer overhead. This module also aborted on every multi-rank run, with no PETSc error message (not root-caused).

WHAT IT MEANS

The structure-preserving PIC simulation is **reproducible**: the same answer on two very different supercomputers, two compilers and any rank count. That is a prerequisite for trusting scaling results. Its energy is conserved to about 10^{-5}, but not exactly, which motivates energy-conserving PIC as next work. A real GPU speed-up needs the DMSwarm particle kernels (push, deposit, locate) on the GPU. ex2 as written does not do that.

8 The AI-agent part: generating code that proves it is correct

8.1 The architecture

The multi-agent system follows the 3-layer hierarchy of the design proposal by McInnes et al. (Figure 11). A **Problem Definition layer** takes governing equations and boundary/initial conditions from the science team. A **Workflow Control layer** holds the orchestrator agent, which keeps the workflow state, assigns tasks and routes results, together with shared and persistent memory. An **Agent Execution layer** holds four specialists: **Mathematical Modeling** (turns the request into precise equations), **Numerical Analysis** (chooses discretization, method and solver), **HPC Code Generation** (writes and runs PETSc code with embedded verification), and **Visualization/Analysis** (turns output into evidence and diagnostic signals). Control moves freely between layers as the solution is refined.

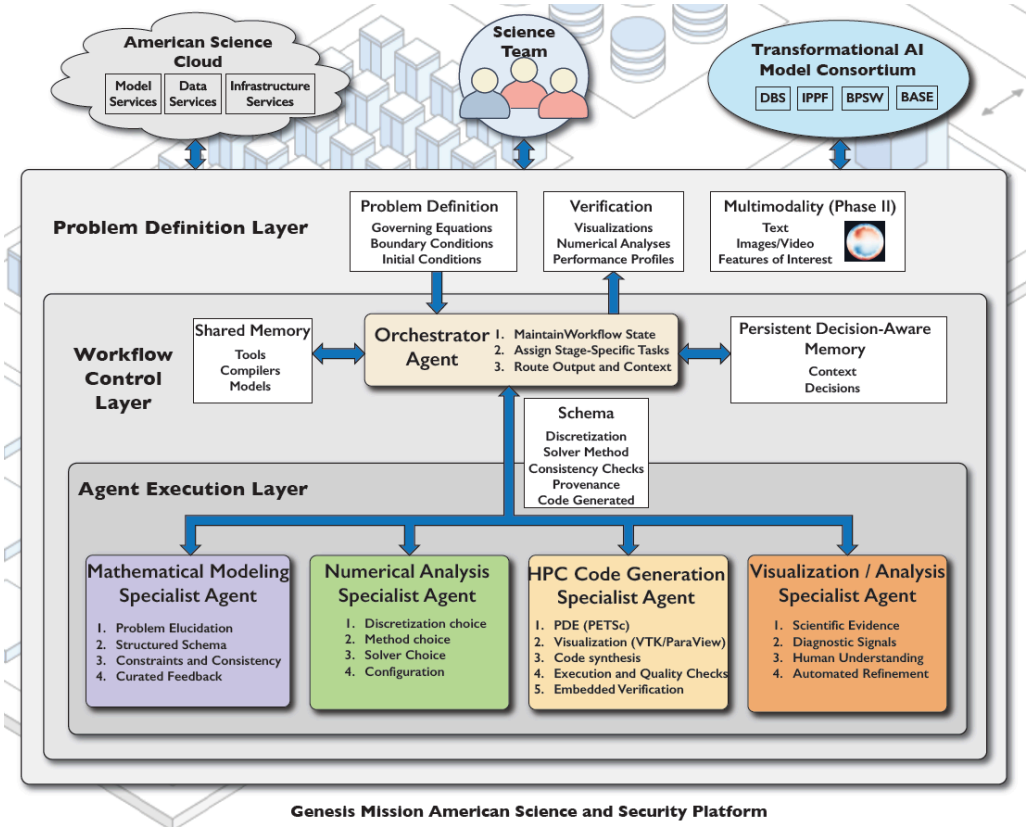


Figure 11. The multi-agent hierarchy. Reproduced from McInnes et al., "Automated Problem-to-Solution Generation for PDE-Based Simulation Science" (accessed 20 April 2026). *What to look at:* the orchestrator in the middle layer connects downward to the four coloured specialist agents. Note the "Verification" box at the top (visualizations, numerical analyses, performance profiles) and "Embedded Verification" listed under HPC Code Generation. This project's `VERIFY{}` contract is a concrete implementation of that idea.

In software, each agent is an **MCP server** from `gitlab.com/petsc/petsc_mcp_servers`, grounded in PETSc documentation via RAG. As surveyed in Phase 1, the orchestrator, PDE-modeling, NA, code-generation and compile-run servers exist. The visualization/analysis agent exists only as a design document.

8.2 Earlier result: agent-generated, verified Grad–Shafranov solvers

This work (the `petsc_mcp_servers_tokamak` repository, July–August 2026) predates the present phase. It shows that the pipeline can produce correct solvers when verification decides acceptance.

- **Session 1 (run of 23 July 2026).** From the prompt "the Grad–Shafranov equilibrium for the plasma in a tokamak", the agents produced a **267-line PETSc `DMDA` + `SNES` solver** with zero hand-written lines. It compiled and ran on 1 and 4 MPI ranks as generated. It verifies itself with the **method of manufactured solutions (MMS)**: choose an exact answer ψ_{exact}, compute the right-hand side it implies, solve, and measure the error. Observed order **p = 2.00**, with max-norm error 8.4×10^{-4} at 33^2 down to 1.3×10^{-5} at 257^2. Total wall time 450.8 s, 21 LLM loops and 5 tool calls in code generation.
- **Session 2.** The built-in orchestrator ran end to end and produced a **DMPlex + PetscFE** solver that converged. Its first run tripped a hard-coded message cap (`cntlimit`) that **mislabelled a successful run as a failure**. This was diagnosed and fixed, and the fix was packaged with others as patches 0001–0004 for the upstream servers.
- **Session 5 (run of 5 August 2026).** One agent-generated, parameterised solver (252 lines, 0 hand-written, 311 s wall, 25 LLM loops / 6 tool calls) reproduced three shaped Cerfon–Freidberg Solov'ev equilibria, each verified at p = 2.00 on 1 and 4 ranks. The safety-factor profile $q(\psi)$ of each agreed with the independent **FreeGS** code to **within 0.2 %**.

Equilibrium	Observed order p	Finest max-norm error	q_0	q_{95}	Elongation κ	Triangularity δ
ITER-like	2.00, 2.00, 2.00	1.12e-7	1.687	2.866	1.691	0.316
NSTX-like	2.00, 2.00, 2.00	2.80e-6	1.510	12.354	2.021	0.336
X-point	2.00, 2.00, 2.00	1.25e-6	1.639	3.190	1.700	0.347

Table 5. Shaped equilibria from one agent-generated solver (`artifacts/run-20260805-155724/metrics.md`). Elongation κ measures how tall the cross-section is relative to its width, and triangularity δ how D-shaped it is.

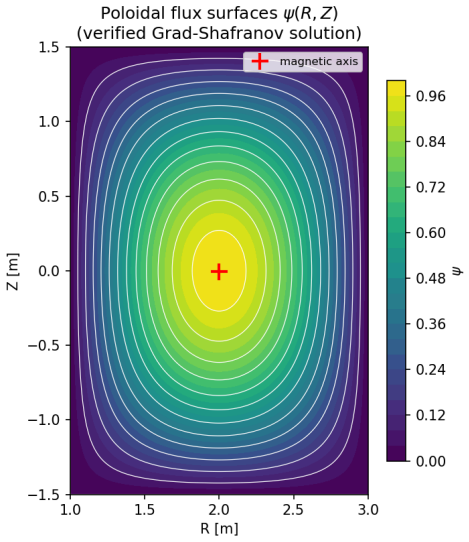


Figure 12. Flux surfaces from the agent-generated DMDA+SNES solver (Session 1). Colours show $\psi(R, Z)$, and white lines are contours, i.e. flux surfaces. The red cross marks the magnetic axis. *What to look at:* the smooth nested contours. Note that this is the manufactured test solution on a rectangle, used for verification, and not a shaped tokamak equilibrium.

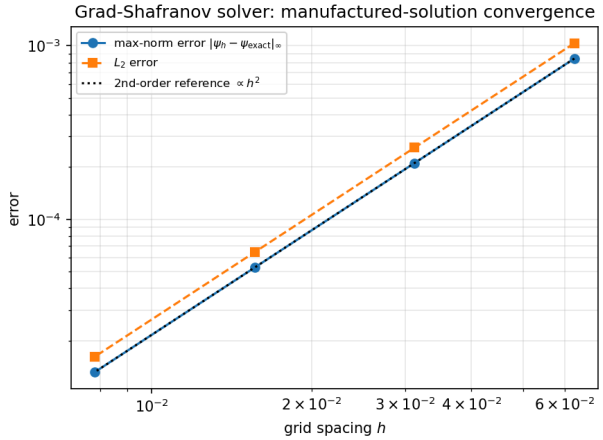


Figure 13. Convergence of the same solver. Error versus grid spacing h, log–log. *What to look at:* both error measures (blue max-norm, orange L_2) fall on straight lines parallel to the dotted h^2 reference. Halving h cuts the error by 4, which is second order, as designed.

8.3 New in this project: the `VERIFY{}` conservation contract

Idea. Turn "the code must conserve X" into something a machine can check. Every generated code must print one line of JSON, for example:

```
VERIFY {"code": "pusher", "method": "boris", "field": "solovev", "steps": 50000,
        "invariants": {"energy_rel_drift_max": 3.171e-14, "p_phi_rel_drift_max": 4.618e-06},
        "finite": true}
```

Implementation (local repository copy `petsc_mcp_servers_verify`, branch `grokbot/verify-contract`, not pushed):

- `verify_contract.py` parses the line, and its `check()` compares each invariant (energy drift, p_φ drift, $\nabla \cdot \mathbf{B}$ maximum, finiteness) against a per-invariant tolerance.
- The compile-run MCP server gains a `run_and_verify` tool. The code-generator prompt now requires the VERIFY line and loops until verification passes. This is a step beyond the original check, which was only "it compiles".

Offline test (`test_verify_contract.py`, using real pusher runs of 50,000 steps):

Code	Field	Energy drift	Verdict
Boris	Solov'ev	3.2e-14	**PASS**
TSDISCGRAD	Solov'ev	1.9e-11	**PASS**
RK4	Solov'ev	1.5e-2 > tolerance 1e-6	**FAIL**
TSBASICSYMPLECTIC	Uniform B	non-finite	**FAIL**
`ct_stag` (CT solver)	Field loop	div B = 3e-24	**PASS**

Table 6. `VERIFY{}` contract offline test (`RESULTS.md` §4, `verify_contract_test.json`). The RK4 drift here (1.5 %) is smaller than the 19 % of Table 3 because this test ran 50,000 steps instead of 10^6.

WHAT IT MEANS

The contract does exactly what it should on known cases. It accepts the structure-preserving codes and rejects both the drifting RK4 code and the unstable one, even though all four compile and run. In an agent loop, a code generator that first produced an RK4 pusher would be sent back to try again, and only a code that conserves would be accepted. The same check catches parallel bugs: particle count and charge are natural invariants to add (Section 9.1).

CAVEAT

The end-to-end LLM loop was **not** run: no LLM back end is configured outside Argonne's internal network. Only the deterministic contract and the tool code were tested. The planned upgrades to the NA agent (a tool that selects structure-preserving schemes) and the visualization/analysis server remain to be built.

9 Scaling on supercomputers

9.1 First pilot: the code did not scale, and particles went missing

Question. If we keep the work per rank fixed (256 cells, 51,200 particles) and add ranks, does the time per step stay constant?

Setup (Phase 2). Fixed box $x \in [0, 4\pi]$, with the mesh refined as ranks are added. 100 steps, $\Delta t = 0.01$, debug queue, 1–2 nodes.

Machine	Ranks	Nodes	s / step	Efficiency vs 1 rank	Poisson s / step	Locate max/min
Polaris	1	1	0.170	1.00	0.0023	1.0
Polaris	4 / 8 / 16 / 32	1	0.255 / 0.350 / 0.536 / 0.941	0.66 / 0.49 / 0.32 / 0.18	0.005–0.013	–
Polaris	64	2	1.687	0.10	0.027	–
Aurora	1	1	0.128	1.00	0.0021	1.0
Aurora	12 / 24 / 48 / 96	1	0.339 / 0.571 / 1.306 / 2.305	0.38 / 0.22 / 0.098 / 0.056	0.009–0.021	3.1 (12), 18.6 (96)
Aurora	192	2	4.371	0.029	0.040	36.1

Table 7. Phase 2 weak-scaling pilot (`RESULTS.md` §3, selected rows). "Locate max/min" is the ratio of the slowest to the fastest rank's time in `DMPlexLocatePoints`.

Findings.

1. **The field solver scales fine. Particle location does not.** The Poisson solve is only 1–2 % of a step at every size. Nearly all the time goes to `DMSwarmMigrate` → `DMPlexLocatePoints`, which finds the cell containing each particle. Its imbalance rises from 1 to 18.6 (96 ranks) and 36 (192 ranks): a few ranks do most of the work while the others wait.
2. Both machines follow the **same efficiency curve**, so the loss is algorithmic, not a hardware or network effect. Spreading 32 ranks over 2 nodes instead of 1 bought only 5 %.
3. **Silent particle loss, a correctness bug.** With $\Delta t = 0.05$, runs with 48 or more ranks lost 1–35 % of their particles in the first step (Polaris, 64 ranks: 2.98 M of 3.28 M left), and the solver then failed. The cause: the distance a fast particle travels in one step, $|v|_{\max}\Delta t = 0.3$, exceeded the width of a rank's subdomain, $4\pi/P$. DMSwarm migration only hands particles to *neighbouring* ranks, so particles that would skip over a rank were dropped. With $\Delta t = 0.01$ the particle count is exactly conserved up to 192 ranks. This is exactly what a `VERIFY{}` invariant on particle count and total charge would catch.
4. With a fixed box, weak scaling is capped near $P_{\max} \approx L/(|v|_{\max}\Delta t)$. A proper study must **grow the box** with the number of ranks.

9.2 Diagnosis: one rank's search grid covered the whole domain

How point location works. To find which cell a particle is in quickly, each rank builds a **hash grid** (a coarse lookup table). It covers the rank's region with "bins", and each bin lists the few cells overlapping it. Look up the bin, then check those few cells. If the bins are too coarse, many cells share a bin, and each lookup degrades to a slow scan through all of them.

The cause, confirmed three independent ways in Phase 3. PETSc 3.26's `DMPlexCreateGridHash()` sizes each rank's hash box from *all* of that rank's local coordinates. On a periodic 1-D mesh, the rank owning the wrap-around cell (the last rank) also holds the vertex at $x = 0$. Its hash box therefore spans the **whole domain** $[0, L]$, but it still gets only about $0.8 \times 256 \approx 204$ bins. Its 256 cells fall into one or two bins, and every particle that leaves its previous cell costs a linear scan of about 256 cells instead of about 1.

1. `-dm_plex_print_locate` on a 4-rank test: ranks 0–2 report hash boxes equal to their own subdomain. Rank 3 reports $(0 - 12.566)$, the whole domain.
2. Per-rank profiling on Aurora (job 8922161, 96 ranks): `DMPlexLocatePoints` took **131 s on rank 95 versus a median of 7.4 s** (18.6×), and rank 95 did 19× the arithmetic of any other rank. Every other rank waits for it at the next collective, so the whole step runs at rank 95's speed.
3. The predicted imbalance grows with P (about $256P/204$ cells per bin on the wrap rank) until a single bin holds all 256 cells. This matches the saturation at **≈41×** seen at 384–1,536 ranks.

It is not the initial particle distribution, not periodic coordinate handling itself, and not migration message volume (messages are small and balanced). Turning the hash off (brute-force search) is worse: 5.3 s per step at 96 ranks.

9.3 The fix

A rejected first fix. The global option `-dm_plex_hash_box_faces 2·256·P` gives every rank fine bins. It worked at 96 ranks (8.2× faster), but the cost of building the hash grows faster than linearly with the number of bins (development-machine test: 4k bins 0.3 s, 65k bins 8 s, 262k bins 145 s). At 1,536 ranks (786k bins on *every* rank) setup never finished. Job 8922177 was cancelled after about 10 minutes on 16 nodes, wasting 2.72 node-hours.

The adopted fix is a 27-line local patch to ex2 (`phase3/patch/ex2_gb_wrap_hash.patch`, not pushed upstream). It adds a new option `-gb_wrap_hash_factor f`. After the mesh is created, any rank whose local bounding box is more than 4× wider than its owned cells (only the wrap rank) requests $f \cdot \text{width}/h$ hash bins in its own options, so its bins are about one cell wide ($f = 2$ used). All other ranks keep the default. The better upstream fix would be for `DMPlexCreateGridHash()` to size the box from the coordinates of owned cells only. That is worth reporting to the PETSc developers.

Effect at 96 ranks (one Aurora node): time per step **2.29 s → 0.28–0.30 s** (7.7–8.2× faster). Locate work imbalance (max/min arithmetic) **19× → 1.07–1.17×**.

9.4 Weak scaling after the fix

Setup (Phase 3). 1D-1V Landau damping, $k = 0.5$, 51,200 particles and 256 cells per rank, $\Delta t = 0.05$, 100 steps (20 steps for the slow "before" runs). The box grows with the number of ranks: $L = \pi P$, so each rank owns a width of π, well above $|v|_{\max}\Delta t = 0.3$, and every P that is a multiple of 4 holds a whole number of $k = 0.5$ wavelengths. (The proposal's original rule $L = 4\pi P/48$ would give a subdomain width of 0.26, below 0.3, and particles would be lost.) Aurora: one job in the debug-scaling queue on 16 nodes running 1, 2, 4, 8 and 16 nodes in turn. Polaris: one job on 8 nodes. Time per step is the slowest rank's `TSStep` time.

Aurora (96 ranks/node)	1 rank	96 (1 node)	192 (2)	384 (4)	768 (8)	1,536 (16)
s/step, after fix	0.147	0.299	0.330	0.342	0.363	0.400
s/step, before fix	0.151	2.29	4.32	4.93	4.85	4.88
Efficiency vs 1 node, after	–	1.00	0.91	0.87	0.82	**0.75**
Efficiency vs 1 node, before	–	1.00	0.53	0.46	0.47	**0.47**
Efficiency vs 1 rank, after	1.00	0.49	0.45	0.43	0.41	**0.37**
Efficiency vs 1 rank, before	1.00	0.066	0.035	0.031	0.031	**0.031**
Locate max/min, before → after	–	18.2 → 1.2	35.6 → 1.2	41.5 → 1.2	41.5 → 1.4	41.4 → 1.3
Particles (all conserved exactly)	51,200	4.9 M	9.8 M	19.7 M	39.3 M	**78.6 M**

Table 8. Aurora weak scaling (job 8922217). The "before" efficiencies versus 1 node are computed from the same `weak_scaling_v2.csv` data that `RESULTS.md` uses (e.g. 2.29/4.88 = 0.47 at 16 nodes).

Polaris (32 CPU ranks/node)	1 rank	32 (1 node)	64 (2)	128 (4)	256 (8)
s/step, after fix	0.192	0.269	0.291	0.314	0.337
s/step, before fix	–	0.949	1.721	2.798	4.958
Efficiency vs 1 node, after [before]	–	1.00 [1.00]	0.92 [0.55]	0.86 [0.34]	**0.80 [0.19]**
Efficiency vs 1 rank, after [before]	1.00	0.71 [0.20]	0.66 [0.11]	0.61 [0.07]	**0.57 [0.04]**
Locate max/min, before → after	–	6.9 → 1.2	13.2 → 1.1	22.7 → 1.2	41.4 → 1.1
Particles (all conserved exactly)	51,200	1.6 M	3.3 M	6.6 M	**13.1 M**

Table 9. Polaris weak scaling (job 7738212). It supersedes an earlier partial 1–2-node job (7738293), with which it agrees to within 5 %.

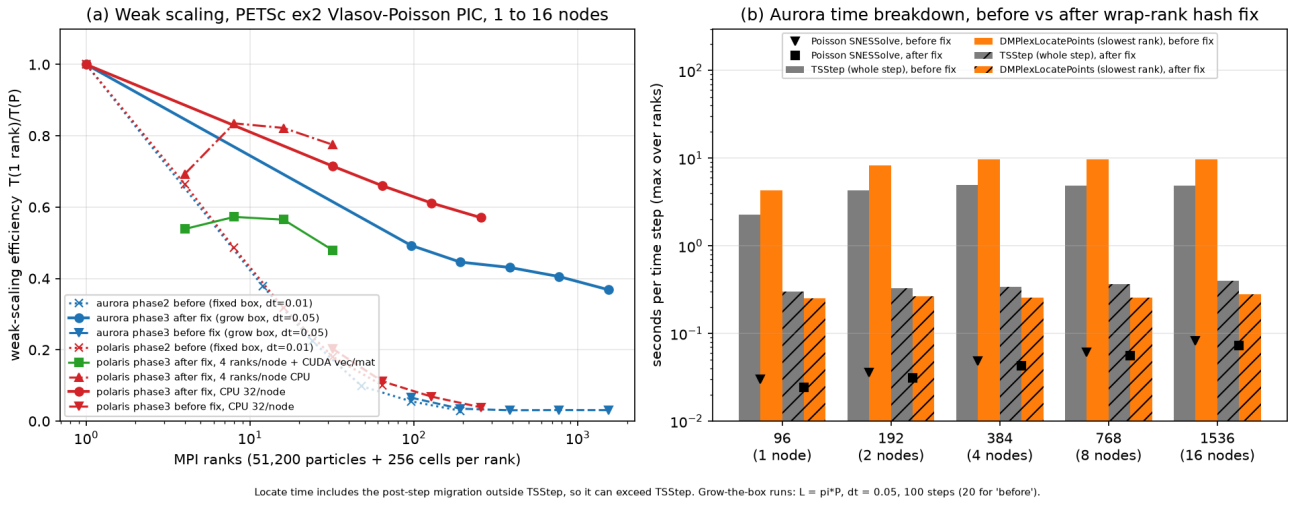


Figure 14. Weak scaling of PETSc ex2 on Aurora and Polaris, before and after the wrap-rank fix. *Panel (a), efficiency versus ranks (log scale), relative to 1 rank.* What to look at: the dotted and dashed "before" curves (×, ▼) plunge to about 0.03–0.04 by a few hundred ranks on both machines. The solid "after" curves stay high: Aurora (blue circles) ends at 0.37 at 1,536 ranks, and Polaris CPU (red circles) at 0.57 at 256 ranks. Most of what remains is the drop from 1 rank to one full node. Green squares (Polaris, 1 rank per GPU with CUDA) sit *below* the matching CPU curve (red triangles). *Panel (b), Aurora time per step by node count (log scale).* Grey and orange solid bars (before) are 2–10 s, dominated by point location on the slowest rank. The hatched bars (after) are about 0.3 s and flat. The black markers, the Poisson solve, are small but grow slowly. Note that locate time includes the post-step migration outside `TSStep`, so it can exceed `TSStep`.

Polaris, 4 ranks per node (1 per A100 GPU). CPU-only: 0.278, 0.230, 0.234, 0.248 s/step at 1, 2, 4, 8 nodes. With CUDA vectors/matrices: 0.357, 0.336, 0.340, 0.401 s/step. The GPU version is **1.29×, 1.46×, 1.46× and 1.62× slower** than the same number of CPU ranks, again because only the field solve is offloaded. (At 4 ranks/node the CPU efficiency relative to 1 node exceeds 1, because one of the four ranks in the 1-node run had slower locate times, which did not repeat at larger counts.)

9.5 What the numbers say, on the same basis

HEADLINE NUMBERS, STATED CAREFULLY

- **Speed-up from the fix:** 12.2× per step at 16 Aurora nodes (4.88 → 0.400 s) and 14.7× at 8 Polaris nodes (4.958 → 0.337 s).
- **Across nodes (baseline = 1 full node):** Aurora 1→16 nodes, efficiency **0.47 before → 0.75 after**. Polaris 1→8 nodes, **0.19 → 0.80**.
- **Overall (baseline = 1 rank):** Aurora at 1,536 ranks **0.03 → 0.37**. Polaris at 256 ranks **0.04 → 0.57**.
- "0.03 → 0.75" would mix two baselines and is *not* a valid comparison.

What is left. (i) A drop *within* one node, from 1 rank to a full node: 0.49 on Aurora and 0.71 on Polaris. Per-rank point-location time roughly doubles (14 s → 25 s per 100 steps) even though the work is balanced. That points to shared memory bandwidth and cache, not imbalance. (ii) A slow rise from 1 to 16 nodes, mostly from the Poisson solve (`SNESSolve` 0.024 → 0.073 s/step, GAMG on a 393k-cell 1-D mesh) and mass-matrix assembly. Point location is now flat with P.

9.6 Conservation at every scale

Global particle count and total weight (charge) are **identical at the first and last step in every run**: 1 to 1,536 ranks on Aurora and 1 to 256 ranks on Polaris (CPU and GPU), with no particle loss. The relative total-energy drift is the same at every P: 1.14×10^{-5} after 100 steps (1.12×10^{-5} for the 20-step runs). The 1-rank run has a different box ($L = \pi$) and gives 1.1×10^{-7}.

WHAT IT MEANS

Splitting the work over up to 1,536 processes does not change the physics. The invariants are preserved independently of the parallel decomposition, which is the "structure preservation at scale" message. The scaling bottleneck was found by measuring, explained from the source code, confirmed by three independent tests and removed with a targeted fix. After the fix the code scales across nodes with 75–80 % efficiency over the range tested in Phase 3 (16 Aurora / 8 Polaris nodes). Phase 4 extended this to 64 Aurora nodes (Section 10.2).

CAVEATS

The patch handles only 1-D meshes (`cdim == 1`). Other DMSwarm codes on periodic Plex meshes in 2-D and 3-D probably have the same wrap-rank problem. The runs are short (100 steps, $t = 5$), and times exclude setup. At 1,536 ranks the wrap rank spends roughly 20 s once, at startup, building its fine hash. The "before" points in Figure 14(a) come from two setups, Phase 2 (fixed box, $\Delta t = 0.01$) and Phase 3 (grow box, $\Delta t = 0.05$), which show the same pathology. All of this is CPU computing. The largest run in Phase 3 is 16 nodes; Phase 4 reached 64 nodes, still not thousands.

10 Phase 4: stronger tests of scale and of time

Phases 2 and 3 answered the first questions: the structure-preserving methods work, and after one targeted fix the particle-in-cell code scales across 16 Aurora nodes. Phase 4 (12 October 2026) asked three harder questions. **(a)** If we keep one problem fixed and give it more nodes, how much faster does it get? **(b)** Does weak scaling keep holding beyond 16 Aurora nodes? **(c)** Do the conservation properties hold over runs 10 to 100 times longer than before? All ALCF jobs in this phase finished normally, and **no particle was lost in any run** (the particle count at the last step equals the count at the first step). The code is the same patched ex2 as in Phase 3, run on CPUs.

STRONG VERSUS WEAK SCALING, IN PLAIN WORDS

Imagine painting a fence. **Strong scaling** asks: if one fence takes one painter 10 hours, how long do 10 painters take? Ideally 1 hour, but in practice painters get in each other's way and spend time coordinating, and once each has only a metre of fence the coordination dominates. **Weak scaling** asks a different question: if every painter gets their own 10-metre stretch, so that 10 painters paint a 100-metre fence, does the job still take 10 hours? Here the work per painter stays the same, and the only extra cost is coordination across a longer fence.

For a simulation, the "painters" are MPI ranks and the "fence" is the simulated domain. Strong scaling shows how fast a fixed problem can be solved. Weak scaling shows how large a problem the machine can handle in the same time. Both are reported against a baseline of **one full node** here.

$$S(N) = \frac{T(1\ \text{node})}{T(N\ \text{nodes})}, \qquad E_{\text{strong}}(N) = \frac{S(N)}{N}, \qquad E_{\text{weak}}(N) = \frac{T(1\ \text{node})}{T(N\ \text{nodes})}\ (\text{work per rank fixed}).$$

> S: speed-up. E: efficiency, 1 is perfect. In strong scaling the problem is fixed, so the ideal speed-up on N nodes is N. In weak scaling the problem grows with N, so the ideal time stays constant and the ideal efficiency is 1. "Wall" times are the whole run, including setup. "TSStep" times count only the time stepping, which is the cleaner measure of how the method scales.

10.1 Strong scaling: one fixed problem on more nodes

Setup. One fixed Landau-damping problem: 49,152 cells in space × 200 particles per cell = **9,830,400 particles**, $\Delta t = 0.05$, 100 steps. It was run on 1, 2, 4, 8 and 16 Aurora nodes (96 ranks per node) and 1, 2, 4 and 8 Polaris nodes (32 ranks per node). The reference is 1 node, because a 1-rank run would take about 45 minutes.

Machine	Nodes (ranks)	Wall s	Speed-up	Efficiency (wall)	TSStep s	Efficiency (TSStep)
Aurora	1 (96)	88.63	1.00	1.00	57.14	1.00
Aurora	2 (192)	51.55	1.72	0.86	32.44	0.88
Aurora	4 (384)	30.66	2.89	0.72	18.70	0.76
Aurora	8 (768)	21.21	4.18	0.52	12.49	0.57
Aurora	16 (1,536)	19.14	**4.63**	**0.29**	10.96	0.33
Polaris	1 (32)	235.24	1.00	1.00	152.81	1.00
Polaris	2 (64)	129.50	1.82	0.91	82.50	0.93
Polaris	4 (128)	68.99	3.41	0.85	42.82	0.89
Polaris	8 (256)	43.70	**5.38**	**0.67**	26.41	0.72

Table 10. Strong scaling of a fixed 9.83-million-particle problem (`phase4/data/strong_scaling.csv`). Speed-up and efficiency are relative to 1 node of the same machine.

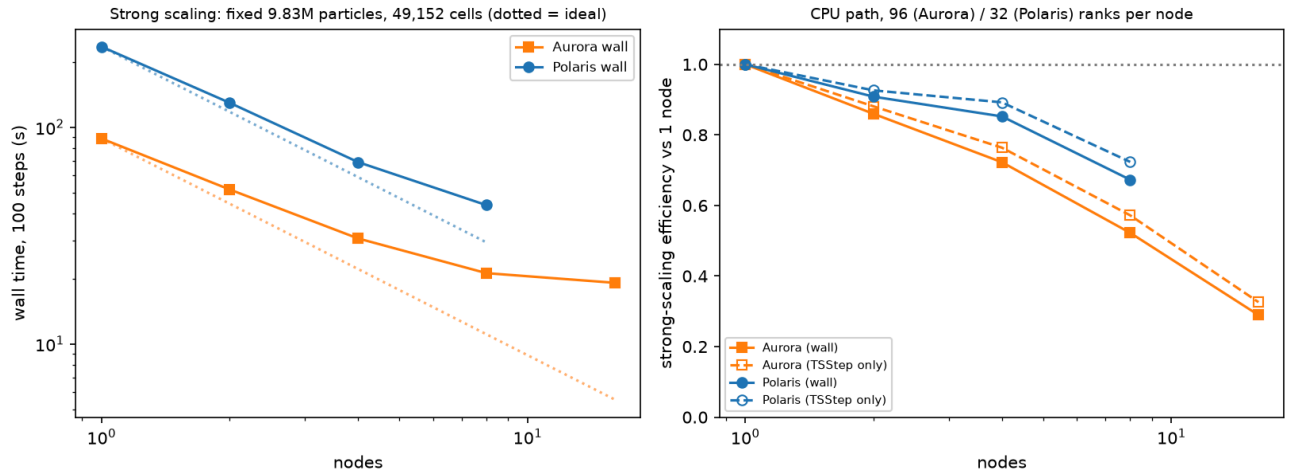


Figure 15. Strong scaling on Aurora (orange) and Polaris (blue). *Left:* wall time for 100 steps versus nodes, both on log scales. The dotted lines are the ideal: twice the nodes, half the time. *What to look at:* both machines start close to the ideal and then bend away from it. Aurora's curve almost flattens between 8 and 16 nodes. *Right:* efficiency versus nodes. Solid lines use wall time, dashed lines only the time-stepping. Polaris stays above 0.67 at 8 nodes. Aurora falls to about 0.3 at 16 nodes.

Why Aurora flattens. Aurora puts 96 ranks on every node, three times as many as Polaris. At 16 nodes the fixed problem is cut into 1,536 pieces, and each rank holds only **32 cells and 6,400 particles**. With so little work per rank, the fixed costs of each step (exchanging particles with neighbours and the global Poisson solve, which needs every rank to take part) dominate, so going from 8 to 16 nodes gains only 10 %. Polaris, with 32 ranks per node, still has about 6× more work per rank at 8 nodes and scales better. This is a property of the problem size, not a fault: a larger fixed problem would strong-scale further. Per node, Aurora is **2.65× faster** than Polaris on this problem (88.6 s versus 235.2 s on 1 node). Wall times include about 30 s of setup (mesh and particle creation) on 1 Aurora node, which does not shrink much with more nodes. That is why the TSStep efficiencies are somewhat higher.

10.2 Weak scaling to 64 Aurora nodes

Setup. The same per-rank load as Phase 3 (256 cells and 51,200 particles per rank, $\Delta t = 0.05$, 100 steps), now on 32 and 64 Aurora nodes. The 32-node job also re-ran a fresh 1-node reference: its TSStep time was 29.60 s, against 29.88 s in Phase 3, an agreement within 1 %. So the new points can be joined to the Phase 3 curve.

Aurora nodes (ranks)	Particles	Wall s	Efficiency (wall)	TSStep s	Efficiency (TSStep)
1 (96)	4.9 M	46.68	1.00	29.60	1.00
32 (3,072)	157.3 M	74.35	0.63	47.33	0.63
64 (6,144)	**314.6 M**	85.73	0.54	50.78	**0.58**

Table 11. Phase 4 weak-scaling runs on Aurora (`phase4/data/weak_scaling_p4.csv`). Combined with Phase 3, the TSStep efficiency versus 1 node is **1.00 / 0.90 / 0.87 / 0.82 / 0.75 / 0.63 / 0.58** at 1, 2, 4, 8, 16, 32 and 64 nodes. (Table 8 lists 0.91 at 2 nodes because it uses the time per step from the Phase 3 profile; `RESULTS.md` rounds the TSStep value to 0.90.)

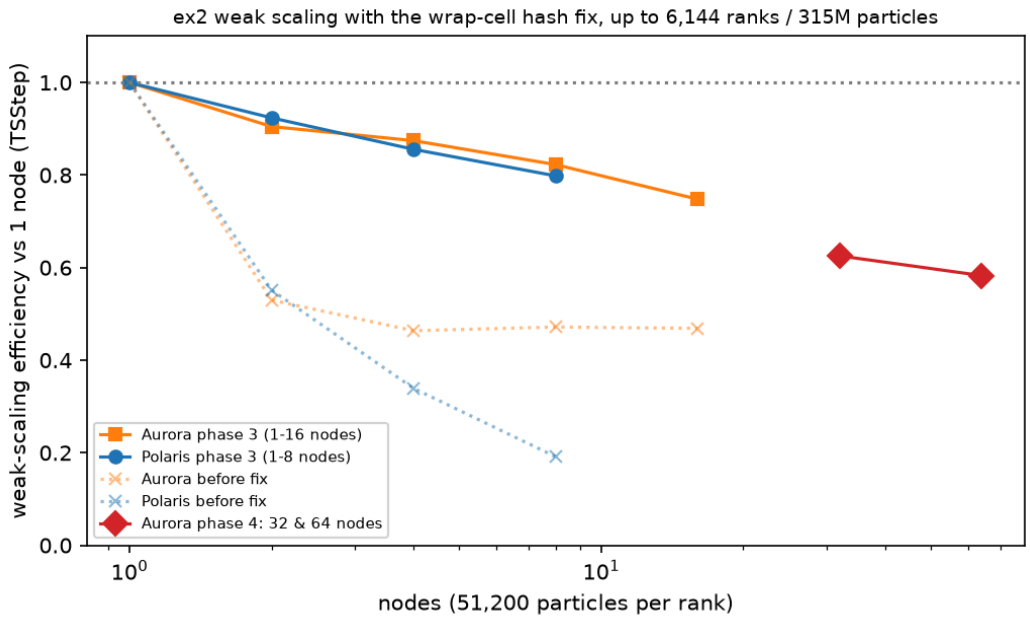


Figure 16. Weak-scaling efficiency versus nodes (log scale), relative to 1 node, from 1 to 64 Aurora nodes. *What to look at:* the faded dotted lines are the "before fix" runs of Phase 3, which collapse quickly. The solid orange (Aurora) and blue (Polaris) lines are the Phase 3 runs after the fix. The red diamonds are the new Phase 4 Aurora runs at 32 and 64 nodes. They continue the orange trend downward gently, from 0.75 at 16 nodes to 0.63 and 0.58. The largest run has 6,144 ranks and 315 million particles.

WHAT IT MEANS

The code now runs a 315-million-particle simulation on 6,144 processes and takes only 1.7× longer per step than a single node does with its 4.9 million particles, while losing no particles. The decline is gradual: going from 32 to 64 nodes costs only 7 % of efficiency. A drop from 1.00 towards 0.6 over 64 nodes is common for a code with a global Poisson solve at every step, but it also shows there is still work to do before thousands of nodes.

CAVEATS

Each point is a single run in the debug-scaling queue, on different node allocations, so ±5 % noise is plausible. We have not yet investigated which part of the step grows with node count (the performance logs are saved with the data). The runs are 100 steps long, on CPUs only, and 64 nodes is still far from the abstract's target of thousands of GPU nodes.

10.3 Long runs of the kinetic plasma: numerical heating

Setup. The Landau-damping simulation of Section 7 (1 rank, $k = 0.5$, perturbation $\alpha = 0.01$, `TSBASICSYMPLECTIC`) was run 10 to 100 times longer than before, on both Aurora and Polaris. We track the total energy W (kinetic plus electric field) and its relative change $(W - W_0)/W_0$, which would be zero for a perfect simulation.

WHAT IS NUMERICAL HEATING?

A particle-in-cell code represents a smooth plasma by a finite number of sample particles. That sampling makes the electric field slightly "noisy", like static on a radio. Each particle receives tiny random kicks from this noise. In a scheme that does not conserve energy exactly, those kicks do not cancel out on average: they add a little energy every step, so the simulated plasma slowly **heats up** even though nothing physical is heating it. The fewer particles per cell, the louder the noise and the faster the heating. It is a well-known property of standard (momentum-conserving, not energy-conserving) PIC methods. Energy-conserving PIC schemes, which update particles and field together so the total energy is kept by construction, are designed to remove it.

Run	Particles	Δt, steps, end time	Max $\lvert\Delta W/W_0\rvert$ Aurora / Polaris	Behaviour	Fitted γ
Medium step	51,200	0.05, 6,000, $t = 300$	4.39×10^{-3} / 4.40×10^{-3}	flat near 10^{-4} until $t \approx 70$, then steady growth (about 1.9×10^{-5} per time unit)	−0.147
Small step	51,200	0.01, 6,000, $t = 60$	3.77×10^{-6} / 3.77×10^{-6}	**bounded** between 2 and 3.8×10^{-6}, no trend	(−0.253, see text)
Few particles, very long	3,200	0.05, 60,000, $t = 3000$	0.818 / 0.805	**strong heating**, about 3×10^{-4} per time unit	−0.123

Table 12. Long-run Landau damping (`phase4/data/phase4_analysis.json`). Linear theory gives $\gamma = -0.1533$.

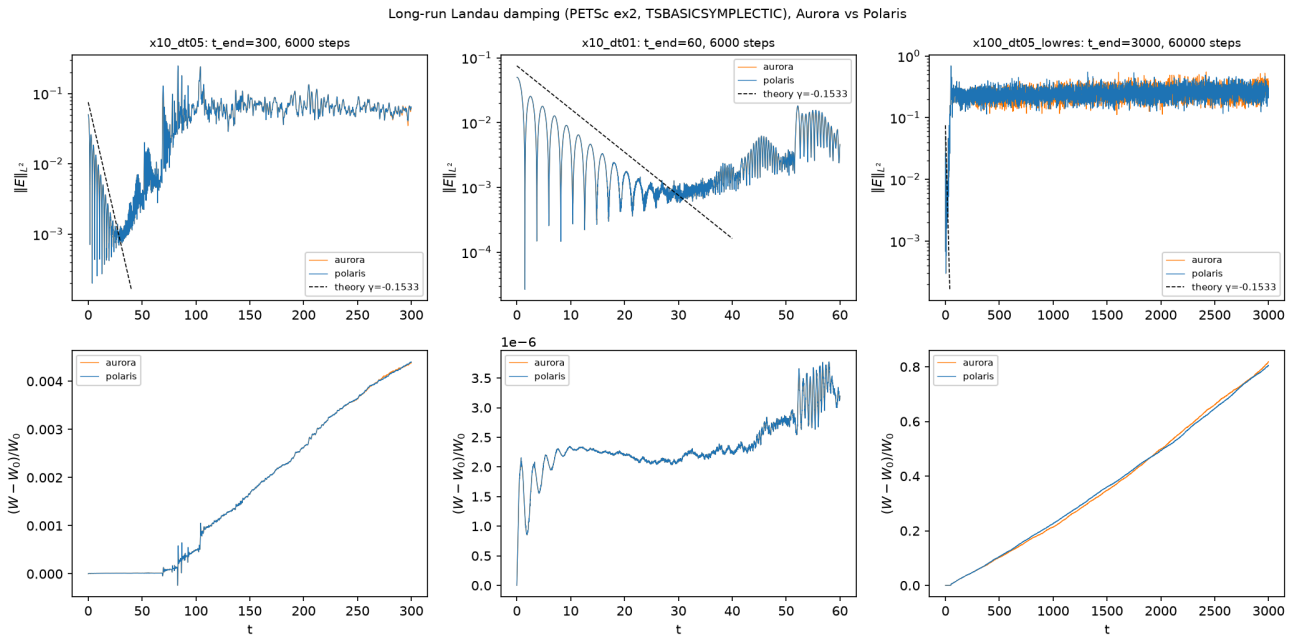


Figure 17. Long-run Landau damping on Aurora (orange) and Polaris (blue). Columns: medium step to $t = 300$, small step to $t = 60$, few particles to $t = 3000$. *Top row:* electric-field strength (log scale) with the theoretical damping line (dashed). *What to look at:* the field first decays along the dashed line, then, once it has sunk into the particle noise (around $t \approx 40$), it grows back to a noise level of about 0.06. *Bottom row:* relative energy error. Left: flat until $t \approx 70$, then a straight rising line to 4.4×10^{-3}. Middle: stays in a band of a few 10^{-6} (note the 10^{-6} scale). Right: rises in a straight line to about 0.8, i.e. 80 % more energy than the plasma started with.

- **The energy error is not bounded in long runs.** At $\Delta t = 0.05$, once the Landau wave has damped into the noise, the total energy grows linearly. With 16× fewer particles, the run heats by about 80 % by $t = 3000$. This is the numerical heating described above. **We therefore do not claim that the ex2 PIC energy is bounded over long times.**
- **Over the physically interesting window it is small.** While the wave is damping ($t \leq 60$ at $\Delta t = 0.01$, $t \leq 70$ at $\Delta t = 0.05$) the error stays between 10^{-6} and 10^{-4}, consistent with Section 7. At the smaller step the 6,000-step run shows no trend.
- **Aurora and Polaris.** The $\Delta t = 0.01$ runs agree to 10^{-14} in energy. The $\Delta t = 0.05$ runs agree to every digit early on and then drift apart after $t \approx 100$ (largest energy difference 4.5×10^{-5} of W_0). That is expected: in the nonlinear, noise-dominated phase the dynamics is chaotic and amplifies tiny round-off differences between the two machines' compilers. It is not a bug. The few-particle runs differ by 2 %.
- **Damping rate.** The simple fit over $t \in [0.5, 15.5]$ gives $\gamma = -0.147$ at $\Delta t = 0.05$, against theory −0.1533 (Section 7's 600-step fit gave −0.161). At $\Delta t = 0.01$ the same fit gives −0.25, because the output is written every step and the fit window then includes early transient peaks; only the $\Delta t = 0.05$ value should be read as a damping-rate check.

10.4 Particle pushers for 100 million steps

Setup. The single-particle tests of Section 5, run 100× longer: 10^8 steps with $\Delta t = 0.2$, on the project's development machine, in the uniform field and in the Solov'ev tokamak field.

Field	Method	Max energy error $\lvert\Delta H/H_0\rvert$	Max momentum error	Status
Uniform B	Boris	4.8×10^{-4} (bounded, oscillating)	1.8×10^{-3}	done
Uniform B	RK4	**0.668** (drifts, levels off after ~10^6 steps)	0.232	done
Solov'ev	Boris	1.2×10^{-12}	4.6×10^{-6} (p_φ, bounded)	done
Solov'ev	RK4	**0.278**	2.2×10^{-4}	done
Both	`TSDISCGRAD`	$\leq 9 \times 10^{-10}$ while running	$\leq 5 \times 10^{-7}$	**did not finish (stalled)**

Table 13. Pusher long runs (`RESULTS.md`, Phase 4). In the uniform field the momentum is the parallel momentum; in the Solov'ev field it is the canonical toroidal momentum p_φ.

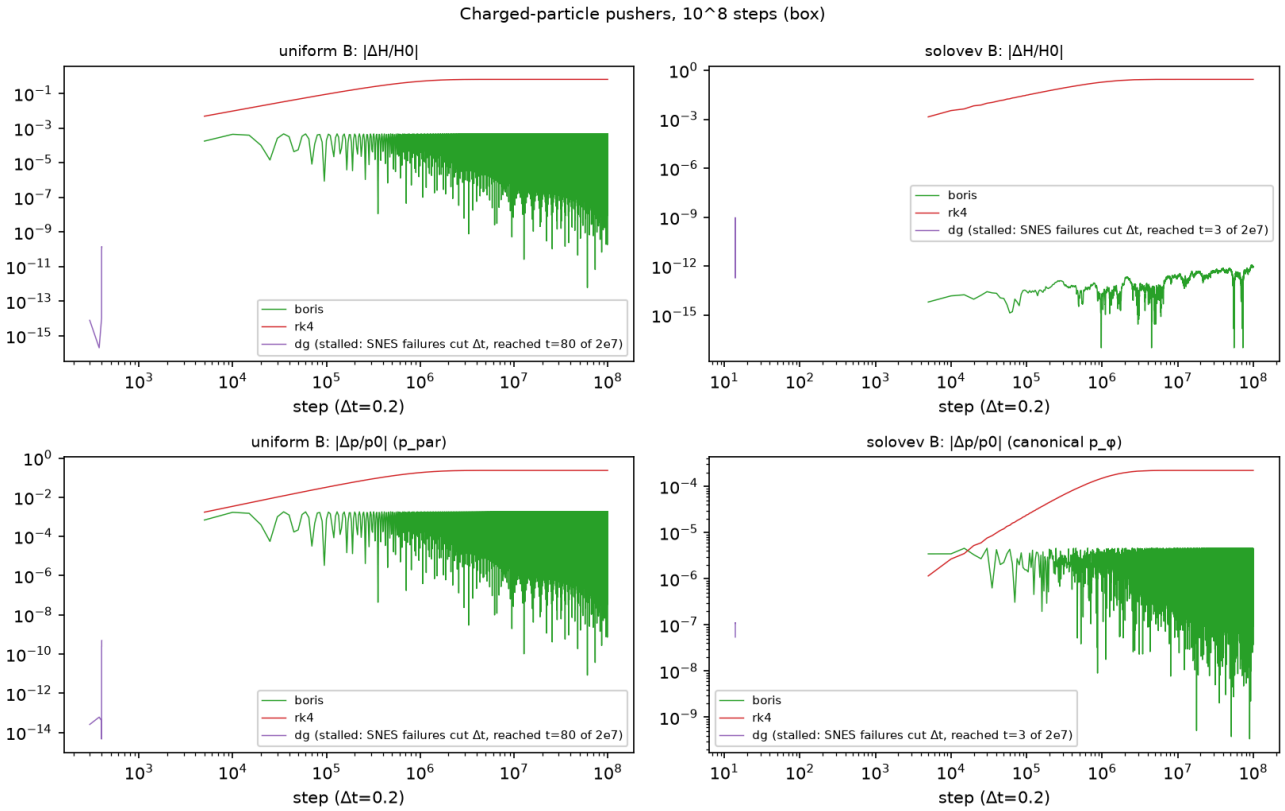


Figure 18. Energy error (top) and momentum error (bottom) over 10^8 steps, uniform field (left) and Solov'ev field (right), log–log. *What to look at:* the red RK4 line climbs steadily and then levels off at a large error. The green Boris band stays flat (it oscillates but never grows) all the way to 10^8 steps. The short purple stub is `TSDISCGRAD`, which stalled very early in the run (see text).

Boris stays bounded for 10^8 steps; RK4 loses 28–67 % of the energy. This is the clean long-time version of the Section 5 result. **`TSDISCGRAD` did not produce a 10^8-step result.** Each of its steps solves a nonlinear equation. When that solve failed, the time stepper was set to retry with a smaller Δt, and repeated failures shrank the step so much that after about 3 hours the uniform-field run had reached only $t \approx 80$ and the Solov'ev run $t \approx 3$, out of $t = 2 \times 10^7$. Its conservation was excellent while it ran, but this is not a long-time result, and we do not claim one.

10.5 Summary of Phase 4

PHASE 4 IN FOUR LINES

- **Strong scaling:** a fixed 9.83 M-particle problem speeds up 4.63× on 16 Aurora nodes (efficiency 0.29, limited by only 32 cells per rank) and 5.38× on 8 Polaris nodes (0.67).
- **Weak scaling:** extended to 64 Aurora nodes, 6,144 ranks and 315 M particles, at 0.58 efficiency versus 1 node, with no particle lost.
- **Long-run PIC:** energy error bounded at 3.8×10^{-6} for $\Delta t = 0.01$ over 6,000 steps, but slow numerical heating in longer runs (0.4 % by $t = 300$; about 80 % with few particles by $t = 3000$). This is the evidence for energy-conserving PIC as the next step.
- **Long-run pushers:** Boris bounded for 10^8 steps; RK4 drifts 28 % (Solov'ev) and 67 % (uniform). The 10^8 `TSDISCGRAD` run stalled.

Phase 4 used about 5.5 node-hours (Section 11).

11 Compute used and acknowledgement

A **node-hour** is one node used for one hour (e.g. 16 nodes for 15 minutes = 4 node-hours). Usage is computed as walltime × nodes from the PBS scheduler records (Phase 4: job runtime from the jobs' own start/done time stamps; the PBS-charged walltime is slightly higher).

Campaign	Machine	Jobs	Node-hours
Phase 3	Aurora	8922147 (1 node, failed: bad symlink) ≈ 0.00; 8922161 diagnostic (1 node, 8:27) 0.14; **8922177 (16 nodes, 10:12, stalled in hash setup, cancelled) 2.72, wasted**; 8922217 sweep (16 nodes, 19:33) 5.21	**8.08**
Phase 3	Polaris	7738204 (cancelled before start) 0; 7738288 (2 nodes, failed: wrong MPI library) ≈ 0.00; 7738293 (2 nodes, 6:54) 0.23; 7738212 (8 nodes, 17:09) 2.29	**2.52**
Phase 3 total (of a 25 node-hour cap), including 2.72 wasted			**10.60**
Phase 4	Aurora	strong scaling (16 nodes × 3.5 min) 0.94; long-run Landau (1 × 22.4 min) 0.37; weak scaling 32 nodes (32 × 2.0 min) 1.08; weak scaling 64 nodes (64 × 1.4 min) 1.53	**3.92**
Phase 4	Polaris	strong scaling (8 nodes × 8.0 min) 1.06; long-run Landau (1 × 32.6 min) 0.54	**1.60**
Phase 4 total (of a 40 node-hour cap; worst case planned 34.7)			**≈ 5.5**
Phase 2 pilots	Polaris	7737974, 7738002, 7738042 (debug, 1–2 nodes)	1.27
Phase 2 pilots	Aurora	13 debug jobs, 1–2 nodes, incl. GPU job 8922008	1.66

Table 14. ALCF usage (`RESULTS.md`). Phases 2 and 3 used 13.53 node-hours and Phase 4 about 5.5, so the whole project used about **19.1 node-hours**. (The 10^8-step pusher runs ran on the development machine and used no ALCF time.) For scale, the plan recorded about 4,964 (Polaris) and 11,999 (Aurora) node-hours remaining in the allocation.

Acknowledgement. This research used resources of the Argonne Leadership Computing Facility (ALCF) at Argonne National Laboratory, through the Director's Discretionary allocation **DD_plasma_physics** on the Aurora and Polaris supercomputers.

12 Limitations: an honest list

Area	Limitation
AI agent loop	The `VERIFY{}` contract and the `run_and_verify` tool were tested **offline** on real pusher and CT runs. **No end-to-end loop with a live LLM was run** in this project, because no LLM back end is configured outside Argonne's internal network. The NA-agent structure-preserving tool and the visualization/analysis server are not yet built (the latter exists as a design document only). The agent-generated solvers shown (Section 8.2) are from earlier work and cover the steady Grad–Shafranov equation only.
Particle pushers	One particle and one orbit class per field. `TSDISCGRAD` accuracy is limited by solver tolerances, and it costs 240–355 s per 10^6 steps versus 0.9 s for RK4 (several hundred times more), because it solves a nonlinear system every step. Its 10^8-step run **stalled**: nonlinear-solver failures made the time stepper shrink Δt again and again, so it reached only $t \approx 80$ (uniform) and $t \approx 3$ (Solov'ev) of $t = 2 \times 10^7$. There is no 10^8-step result for it. The 10^8-step Boris and RK4 runs used a single particle on the development machine.
`TSBASICSYMPLECTIC`	Fails (blows up within 2,000 steps) with $\mathbf{v} \times \mathbf{B}$ forces, because it assumes a separable Hamiltonian. Usable only for electrostatic problems.
Divergence-free solver	2-D **kinematic** constrained transport only (prescribed flow, no magnetic feedback on the flow). DMSTAG version run serially only. 3-D and the finite-element (FEEC) alternative were not attempted.
Kinetic PIC	1D-1V only. Physics runs of 600 steps (up to 16 ranks on Polaris and 12 on Aurora). Scaling runs are only **100 steps** (20 for "before" runs). Total energy is conserved to about 10^{-5} of the total, but not exactly (6–7 % of the peak field energy). The fitted damping rate is about 5 % steeper than linear theory (−0.161; −0.147, about 4 % shallower, in the 6,000-step run). **The energy error is not bounded in long runs**: at $\Delta t = 0.05$ it grows linearly after $t \approx 70$ to 4.4×10^{-3} at $t = 300$, and a 3,200-particle run heats by about 80 % by $t = 3000$ (numerical heating of a non-energy-conserving PIC scheme). Only the small-step run ($\Delta t = 0.01$, $t \leq 60$) is bounded, at 3.8×10^{-6}. Long runs are 1 rank only.
GPUs	All scaling runs used CPUs. The **GPU path was slower**: 1.35× (Polaris pilot) and 1.3–1.6× (Polaris scaling) slower than CPU, and 11.7× slower on one Aurora tile, because ex2 offloads only the small field solve and particles stay on the host. The Aurora `petsc/3.25.2-sycl` module aborts at more than 1 rank (not root-caused).
Scale	Largest run: 64 Aurora nodes (6,144 CPU ranks, 315 M particles) and 8 Polaris nodes. This is far from the abstract's target of "thousands of GPU nodes". Weak-scaling efficiency versus 1 node falls to 0.58 at 64 nodes. Each scaling point is a single run (±5 % noise plausible), and the cause of the slow decline beyond 16 nodes was not investigated.
Strong scaling	A fixed 9.83 M-particle problem reaches only 4.63× on 16 Aurora nodes (efficiency 0.29), because each rank holds just 32 cells. Larger fixed problems were not tried. Wall times include about 30 s of setup.
Load-imbalance fix	The ex2 wrap-hash patch is **local and not upstreamed** to PETSc. It handles 1-D meshes only. The root cause sits in `DMPlexCreateGridHash()` and probably affects 2-D/3-D periodic meshes too. A first attempted fix wasted 2.72 node-hours.
Remaining inefficiency	Efficiency drops to 0.49 (Aurora) and 0.71 (Polaris) within a single node (memory bandwidth/cache), and the Poisson solve grows slowly with node count.
Code status	The `VERIFY{}` branch (`grokbot/verify-contract`) and the ex2 patch are not pushed. Project storage quotas on ALCF were effectively zero at survey time, so runs worked from home directories.

13 Next steps

1. **Energy-conserving PIC.** Couple particles and field implicitly (discrete-gradient or implicit-midpoint coupling), so total energy, not just the particle part, is conserved. This is now backed by our own measurements: the long runs in Section 10.3 show steady numerical heating once the wave has damped (0.4 % by $t = 300$; about 80 % by $t = 3000$ with 3,200 particles). It also addresses the 6–7 % of peak field energy drift in Section 7. A rerun of the same long tests would show directly whether the new scheme removes the heating.
2. **GPU-resident particles.** Move DMSwarm push, deposit and locate onto the GPU. Without this, GPUs cannot speed up ex2.
3. **End-to-end LLM agent loop.** Connect an LLM back end and run the planned four-prompt experiment: a Boris pusher in a tokamak field, Vlasov–Poisson Landau damping with a symplectic integrator, CT induction, and Maxwell with FEEC. Record pass rate, LLM loops, tool calls and human-edited lines, including the demonstration "agent writes RK4 → analysis flags drift → rollback → symplectic method". Also build the NA-agent structure-preserving tool, the visualization/analysis server and the missing agent cards.
4. **Larger runs on Aurora.** Weak scaling now reaches 64 nodes at 0.58 efficiency. Profile what grows between 16 and 64 nodes (the logs are saved), then extend toward the plan's 1,024-node (and optionally 2,048-node) points before the allocation ends on 7 November 2026. Large-queue jobs need submitting by about 30 October.
5. **3-D and FEEC.** Extend constrained transport to 3-D and to full MHD with flow feedback, or use the finite-element exterior calculus pairing (Nédélec / Raviart–Thomas elements in PETSc).
6. **Upstream.** Report the `DMPlexCreateGridHash()` wrap-rank issue to PETSc with a general (2-D/3-D) fix. Add particle count and charge to the `VERIFY{}` contract. Push the contract branch.
7. **Discrete-gradient long runs.** Find why the `TSDISCGRAD` nonlinear solves fail in the 10^8-step runs (tolerances, initial guess, step size) and repeat the run, so that a long-time result exists for it as well.
8. **Practicalities.** Request project storage quotas on ALCF, and keep each claim in the talk tied to the measured baseline (Section 9.5).

14 Glossary

Axisymmetric. The same at every angle around a central axis. Tokamak fields are (nearly) axisymmetric, which makes p_φ conserved.

Boris pusher. The standard particle-pushing method in plasma codes: half electric kick, exact magnetic rotation, half electric kick. Preserves speed in pure magnetic fields and phase-space volume.

Canonical toroidal momentum p_φ. $Rv_\varphi + \frac{q}{m}\psi$. Conserved by axisymmetry and linked to particle confinement.

Collective (MPI). An operation in which all ranks take part, so all must reach it before any continues.

Constrained transport (CT). Updating a magnetic field on a staggered grid so that its discrete divergence stays exactly zero.

Discrete gradient (`TSDISCGRAD`). An implicit integrator that conserves energy exactly (to solver tolerance) by using an averaged gradient. We use the Gonzalez version.

Divergence, $\nabla \cdot \mathbf{B}$. The net outflow of a field from a tiny volume. Zero for magnetic fields (no monopoles).

DMDA / DMSTAG / DMPlex / DMSwarm. PETSc mesh managers for structured grids, staggered grids, unstructured meshes and particle clouds, respectively.

Domain decomposition. Splitting the simulated region into pieces, one per rank.

Efficiency (weak scaling). $T(P_{\text{ref}})/T(P)$, time per step at the baseline divided by time per step at P ranks. Always state the baseline.

Efficiency (strong scaling). Speed-up divided by the increase in resources: $S(N)/N$ for N nodes against 1 node.

Energy-conserving PIC. A particle-in-cell scheme that advances particles and fields together (e.g. implicitly) so that the total energy is conserved by construction. It removes numerical heating.

FEEC. Finite element exterior calculus: finite elements (e.g. Nédélec, Raviart–Thomas) that preserve div/curl identities.

Field-loop advection. A standard test where a ring of magnetic field is carried by a flow. It is sensitive to $\nabla \cdot \mathbf{B}$ errors.

Flux surface. A nested surface in a tokamak on which magnetic field lines lie. Contours of ψ.

FreeGS. An independent Grad–Shafranov equilibrium code, used as a cross-check.

Fusion. Light nuclei merging into a heavier one and releasing energy, e.g. deuterium + tritium → helium + neutron.

GAMG. PETSc's algebraic multigrid preconditioner, used to speed up the Poisson solve.

GPU / tile. Graphics processor, a massively parallel arithmetic engine. Aurora's GPUs have 2 "tiles" each (12 per node).

Grad–Shafranov equation. The equation for force-balance equilibrium of an axisymmetric plasma, for the flux $\psi(R, Z)$.

Gyrofrequency ω_c. $|q|B/m$, the angular frequency at which a charged particle circles a magnetic field line.

Hamiltonian. The energy function H that generates the motion of a conservative system via $du/dt = S\nabla H$.

Hash grid (point location). A coarse lookup table of "bins" used to find quickly which mesh cell contains a point.

Integrator (time integrator). A recipe for advancing the state of an ODE by one time step.

Invariant. A quantity that the exact dynamics keeps constant (energy, momentum, phase-space volume, $\nabla \cdot \mathbf{B} = 0$, particle number).

Jacobian determinant ($\det J$). The volume scaling factor of a map. $\det J = 1$ means phase-space volume is preserved.

Kinematic (induction). The magnetic field is moved by a prescribed flow without acting back on it.

Landau damping. Collisionless decay of a plasma wave due to energy exchange with resonant particles. For $k = 0.5$ the theoretical rate is $\gamma \approx -0.1533$.

LLM, agent, orchestrator. Large language model; an LLM in a loop with tools; an agent that coordinates other agents.

Load imbalance. Unequal work across ranks, so that the slowest rank sets the pace. Measured as max/min across ranks.

Lorentz force. $q(\mathbf{E} + \mathbf{v} \times \mathbf{B})$, the force on a charge in electric and magnetic fields.

Manufactured solution (MMS). A verification method: pick an exact answer, derive the matching source term, and check that the code converges to it at the expected order.

MCP server. A program exposing tools to AI agents via the Model Context Protocol.

MHD. Magnetohydrodynamics, a fluid description of a plasma coupled to magnetic fields.

Migration. Moving particles to the rank that owns the cell they have entered.

MPI, rank. Message Passing Interface, the standard for parallel programs. A rank is one MPI process.

Node, node-hour. One computer within a supercomputer. A node-hour is one node for one hour, the unit of allocation.

Numerical heating. A slow, unphysical rise of the total energy in a particle-in-cell simulation, caused by field noise from the finite number of particles in a scheme that does not conserve energy exactly. Worse with fewer particles per cell.

Order of accuracy p. The error scales like h^p. $p = 2$ means halving h quarters the error.

PETSc. Portable, Extensible Toolkit for Scientific Computation, an open-source parallel solver library.

Phase space. The space of positions and velocities together.

Particle-in-cell (PIC). A kinetic simulation method with particles moving through fields computed on a grid.

Plasma. Ionised gas of free electrons and ions that responds collectively to electromagnetic fields.

Poloidal / toroidal. The short way around a torus (cross-section) / the long way around it.

RAG. Retrieval-augmented generation: grounding an LLM in retrieved documents (here, PETSc documentation).

RK4. The classic fourth-order Runge–Kutta integrator. Very accurate per step, but not structure-preserving.

Safety factor q. Toroidal turns per poloidal turn of a field line. q_0 is at the axis, q_{95} near the edge.

Secular drift. Error that grows steadily with time instead of staying bounded.

Separable Hamiltonian. $H = T(\text{momentum}) + V(\text{position})$. Required by Störmer–Verlet / `TSBASICSYMPLECTIC`.

SNES / KSP / TS. PETSc's nonlinear solvers, linear solvers and time steppers.

Solov'ev equilibrium. An exact analytic solution of the Grad–Shafranov equation. The Cerfon–Freidberg version can be shaped like real machines.

Speed-up. $T(\text{baseline})/T(\text{more resources})$ for the same fixed problem.

Staggered grid. A grid that stores different quantities at different locations (vertices, faces, cells).

Störmer–Verlet (leapfrog). A second-order symplectic integrator for separable Hamiltonians.

Strong scaling. Solving one fixed problem with more ranks or nodes. Ideally N times the nodes gives N times the speed.

Structure-preserving method. A numerical method that exactly preserves a geometric or physical property of the equations.

Symplectic. Preserving the Hamiltonian geometric structure of phase space. Implies volume preservation and bounded long-time energy error for a fixed Hamiltonian system (as for the Boris and pusher tests). It does not by itself make a coupled particle–field code energy-bounded (Section 10.3).

Tokamak. A toroidal magnetic-confinement fusion device with helical field lines.

`VERIFY{}` contract. This project's rule that generated codes print their invariants as JSON, to be checked against tolerances before acceptance.

Vlasov–Poisson. Kinetic equations for a plasma with a self-consistent electrostatic field.

Weak scaling. Growing the problem with the number of ranks, so that work per rank is fixed. Ideally the time per step stays constant.

15 References

Only works and resources named in the project's own sources are listed. Where the sources give no full bibliographic details, none are invented.

1. L. C. McInnes et al., "Automated Problem-to-Solution Generation for PDE-Based Simulation Science", design proposal (DE-FOA-0003612). Accessed 20 April 2026. Source of Figure 11.
2. S. Sharma and M. Knepley, "Generative AI for Scalable and Structure-Preserving Plasma Simulations: A Complex Systems Approach to Nuclear Fusion Energy", abstract, Conference on Complex Systems (CCS) 2026.
3. PETSc, the Portable, Extensible Toolkit for Scientific Computation, version 3.26.0 (and ALCF module 3.25.2). Tutorial `src/ts/tutorials/hamiltonian/ex2.c`. Source routine `DMPlexCreateGridHash()` in `plexgeometry.c`.
4. PETSc MCP servers: `https://gitlab.com/petsc/petsc_mcp_servers`.
5. Tokamak multi-agent project: `github.com/engineer-scientist/petsc_mcp_servers_tokamak` (artifacts `run-20260723-113024`, `run-20260805-155724`, `orchestrator-20260724`). Includes the US-RSE 2026 poster and the Argonne summer-intern presentation (29 July 2026).
6. A. J. Cerfon and J. P. Freidberg, analytic (Solov'ev-type) shaped tokamak equilibria, as used in the tokamak project.
7. FreeGS, an independent Grad–Shafranov equilibrium code, used as an independent cross-check of $q(\psi)$.
8. T. A. Gardiner and J. M. Stone (2005), field-loop advection test, as cited in `sp/ct_stag/ct_stag.c`.
9. Argonne Leadership Computing Facility: Aurora and Polaris supercomputers, allocation DD_plasma_physics.
10. US Department of Energy Genesis Mission (as referenced in the abstract and Figure 11).

Appendix A. Where every number comes from

Content	Source file(s) (under `/workspace`)
Title, authors, goals, Figure 11 citation	`work.txt` (abstract)
Phase 1 survey, gaps, ALCF environment, plan and milestones	`research/PLAN.md`
Pushers, CT, ex2 physics, GPU trials, pilot scaling, VERIFY test, Phase 2 usage	`research/results/RESULTS.md` §1–4; `pusher_summary.csv`; `verify_contract_test.json`
Phase 3 diagnosis, fix, weak-scaling tables, usage	`research/results/RESULTS.md` (Phase 3); `research/phase3/NOTES.md`; `phase3/data/{aurora_p3diag_8922161, aurora_p3ws_8922217, polaris_p3ws_7738212}_reduced.csv`; `weak_scaling_v2.csv`; `phase3/patch/ex2_gb_wrap_hash.patch`
Phase 4: strong scaling, 32/64-node weak scaling, long-run Landau and pusher runs, usage	`research/results/RESULTS.md` (Phase 4); `research/phase4/data/{phase4_analysis.json, strong_scaling.csv, weak_scaling_p4.csv}` and raw job output in `p4_results_aurora/`, `p4_results_polaris/`; analysis script `phase4/analysis/analyze4.py`
Grad–Shafranov agent results	`research/petsc_mcp_servers_tokamak/README.md`; `artifacts/run-20260723-113024/{metrics.md, verification.json, manifest.json}`; `artifacts/run-20260805-155724/metrics.md`
Talk framing, fact guards, next steps	`slides/ccs2026-zshlOY/NOTES.md`, `notes/g1–g4.md`, final deck
Figures	`research/results/figs/*.png`; GS figures in `artifacts/run-20260723-113024/`; photographs and diagrams in `slides/ccs2026-zshlOY/user_images/`

Phase 4 cross-checks. Recomputed from `phase4_analysis.json`: strong scaling Aurora $88.63/19.14 = 4.63$, $4.63/16 = 0.29$; Polaris $235.24/43.70 = 5.38$, $5.38/8 = 0.67$; weak scaling at 64 nodes (TSStep) $29.60/50.78 = 0.58$, particles $6{,}144 \times 51{,}200 = 314.6$ M; Phase 4 usage $3.92 + 1.60 = 5.52$ node-hours, project $13.53 + 5.52 \approx 19.1$.

Cross-checks performed for this report. The weak-scaling efficiencies were recomputed from the reduced CSVs. Aurora at 16 nodes: $2.288/4.881 = 0.469$ (before, vs 1 node), $0.299/0.400 = 0.748$ (after, vs 1 node), $0.147/4.881 = 0.031$ and $0.147/0.400 = 0.368$ (vs 1 rank). Speed-ups: $4.881/0.400 = 12.2$, Polaris $4.958/0.337 = 14.7$. The RK4 drifts were checked against `pusher_summary.csv`: Solov'ev 0.188, uniform 0.518. The figure `results/figs/weak_scaling_v2.png` is byte-identical to the final Phase 3 figure that includes the 8-node Polaris job.